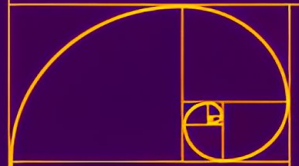
HUNTER
The City University of New York

C++11

CSCI 33500: Software Design and Analysis III

Sadab Hafiz
sh3646@hunter.cuny.edu



05

Initialization

Before C++11

- Initializing variables and objects was completely inconsistent depending on the data type before C++11
- Different syntax for different types
  - Arrays used aggregate initialization
    - `int arr[ ] = {10, 20, 30};`
  - Classes used parentheses
    - `MyClass obj("Example",5);`
  - Local variables used assignment
    - `int num = 5;`
- More critically, using `( )` could accidentally cause the compiler to mistake an object declaration for a function declaration
  - `MyClass default_instance();`
    - Looks like an object creation
    - C++ would interpret this as a function declaration

Initialization

C++11 Uniform Initialization

- C++11 introduced a single, unified way to initialize any variable, object, or container using `{ }`
- This allows consistent syntax across
  - Primitive types:
    - `int num{5};`
  - Aggregates:
    - `MyClass obj{“Example”,5};`
  - STL Containers:
    - `std::vector<int> v{1, 2, 3};`
- It also solves the syntax confusion for function declaration and class object initialization
  - `MyClass default_instance{};`
    - Calls default constructor
  - `MyClass function();`
    - Function declaration

Narrowing Conversions

C++11 Improvement

- Before C++11:
  - `int num = 3.14;`
  - Compiles without any error or warning
  - Silently truncates the value to 3
- C++11 improvement:
  - `int num{3.14};`
  - Using `{ }` forbids narrowing conversions
  - Triggers a **compilation error**
    - No accidental data loss
- The use of `{ }` allows safer coding by catching accidental data truncation at compile time

Initializer List

How does it work?

- C++11 introduced a lightweight type called `std::initializer_list` in the standard library that gives functions and constructors access to an array of temporary objects
- **NOT** the same as Member Initializer List you have seen before while looking at constructors
- For example:
  - `std::vector<int> v{1, 2, 3};`
  - The compiler bundles the passed values into a `std::initializer_list` and hands it to a vector constructor

Initializer List

Custom Braced Constructor

➔ You can define your own custom braced constructor. For example:

```
# include <initializer_list>
# include <vector>
# include <string>
class Books {
    private:
        std::vector<std::string> titles;
    public:
        // constructor that takes a { } list
        Books(std::initializer_list<std::string> books) {
        for (std::string book: books) {
            titles.push_back(book);
        }
    }
};

int main () {
    // example usage
    Books recommendations = {
        "The Silent Patient",
        "And Then There Were None",
        "Gone Girl"
    };
}
```

Constructor

Default Constructor Behavior

- If you don’t write any constructors at all, the compiler automatically generates a default constructor

- If you do write any custom constructor, the compiler doesn’t provide a default constructor
  - Includes any parameterized constructor
  - Includes `std::initializer_list`
  - In this case you can use `=default` for a compiler provided default constructor
    - `Books() = default;`

Constructor

Member Initializer List

```
class Player {
    private:
        std::string name;
        int health;
    public:
        // target constructor (using member initializer list)
        Player(std::string n, int h) : name(n), health(h) {}
        // calls the main constructor with a default health value
        Player(std::string n) : Player(n, 100) {}
};
```

- ➔ Using a member initializer list initializes the data members when they are created, right before the constructor body runs
- ➔ Using the assignment operator is inefficient because it:
  - ◆ Creates default objects
  - ◆ Changes them via assignment
- ➔ Assignment is not allowed for:
  - ◆ A `const` data member
  - ◆ A reference (&) data member
- ➔ C++11 also introduces the ability for a constructor to delegate its work by calling another constructor

Constructor

Implicit Conversions

- ➔ Any constructor that can be called with a single parameter acts as an implicit conversion operator
- ➔ The compiler is legally allowed to silently convert a raw value (`int`) into a temporary instance of your class even if you don’t want it

```
class Classroom {
    private:
        int capacity_;
    public:
        Classroom(int capacity) : capacity_(capacity) {}
};
// a function that expects a full Classroom object
void print_classroom(Classroom current) {
    /* ... */
}
int main() {
    // C++ silently converts the number 25 into a temporary Classroom object
    print_classroom(25);
    // looks like assignment, but it's an implicit conversion
    Classroom cs335 = 25;
}
```

Constructor

Disable Implicit Conversion

- ➔ C++11 introduced the `explicit` keyword, which can be used to disable implicit conversions
- ➔ It forces the compiler to require explicit initialization syntax

```
class Classroom {
    private:
        int capacity_;
    public:
        // adding `explicit` disables silent conversions
        explicit Classroom(int capacity) : capacity_(capacity) { }
};
void print_classroom(Classroom current) { /* ... */ }
int main() {
    print_classroom(25); // compilation error!
    Classroom cs_335 = 25; // compilation error!

    // correct syntax
    Classroom cs_335{25};
    print_classroom(Classroom{25});
}
```

Constructor

Default Values

- ➔ C++11 introduced the ability to add default values for data members
- ➔ This allows assigning data members directly inside the class definition

```
class Classroom {
    private:
        int capacity_ = 25; // initialization
        std::string building_ = "Hunter West";
public:
    // If a user calls the default constructor, it automatically uses 25 and "Hunter West"
    Classroom() = default;
    // This custom constructor overrides the 25, but building_ stays "Hunter West"
    explicit Classroom(int capacity) : capacity_(capacity) {}
};
```

~~Big Three~~

Big Five

Do you Remember: What Special Member Functions Do You Get?

What you get

What you write	default constructor	destructor	copy constructor	copy assignment	move constructor	move assignment
nothing	defaulted	defaulted	defaulted	defaulted	defaulted	defaulted
any constructor	not declared	defaulted	defaulted	defaulted	defaulted	defaulted
default constructor	user declared	defaulted	defaulted	defaulted	defaulted	defaulted
destructor	defaulted	user declared	defaulted (!)	defaulted (!)	not declared	not declared
copy constructor	not declared	defaulted	user declared	defaulted (!)	not declared	not declared
copy assignment	defaulted	defaulted	defaulted (!)	user declared	not declared	not declared
move constructor	not declared	defaulted	deleted	deleted	user declared	not declared
move assignment	defaulted	defaulted	deleted	deleted	not declared	user declared

Howard Hinnant's Table: https://accu.org/content/conf2014/Howard_Hinnant_Accu_2014.pdf
Note: Getting the defaulted special members denoted with a (!) is a bug in the standard.

~~Big Three~~

Big Five

Do you Remember: What Special Member Functions Do You Get?

What you get

What you write	default constructor	destructor	copy constructor	copy assignment	move constructor	move assignment
nothing	defaulted	defaulted	defaulted	defaulted	defaulted	defaulted
any constructor	not declared	defaulted	defaulted	defaulted	defaulted	defaulted
default constructor	user declared	defaulted	defaulted	defaulted	defaulted	defaulted
destructor	defaulted	user declared	defaulted (!)	defaulted (!)	not declared	not declared
copy constructor	not declared	defaulted	user declared	defaulted (!)	not declared	not declared
copy assignment	defaulted	defaulted	defaulted (!)	user declared	not declared	not declared
move constructor	not declared	defaulted	deleted	deleted	user declared	not declared
move assignment	defaulted	defaulted	deleted	deleted	not declared	user declared

Howard Hinnant's Table: https://accu.org/content/conf2014/Howard_Hinnant_Accu_2014.pdf
Note: Getting the defaulted special members denoted with a (!) is a bug in the standard.

Destructor

Freeing Dynamic Memory

- The destructor is responsible for freeing any dynamic memory allocated by the class instance
- The destructor gets called:
  - When an object leaves the scope
  - Directly calling `delete` on a dynamically allocated object
- In C++11, all destructors are marked as `noexcept`, which means they are forbidden from throwing exceptions

```
class Classroom {
    private:
        int capacity_;
        int* seats_;
    public:
        Classroom(int capacity)
            : capacity_(capacity), seats_(new int[capacity]) {}
        // Destructor
        ~Classroom() {
            delete[] seats_;    // deallocate dynamic memory
    }
};
```

Copy and Move

The following slides are provided by Professor Tiziana Ligorio

Recap

```
LinkedBag();
~LinkedBag();     // Destructor
int getCurrentSize() const;
bool isEmpty() const;
bool add(const T& new_entry);
bool remove(const T& an_entry);
void clear();
bool contains(const T& an_entry) const;
int getFrequencyOf(const T& an_entry) const;
std::vector<T> toVector() const;
```

What if we need a copy of the bag?

Copy Constructor

1. Initialize one object from another of the same type

```
MyClass one;
MyClass two = one;
```

More explicitly

```
MyClass one;
MyClass two(one); // Identical to above.
```

Creates a **new object** as a copy of another one

Compiler will provide one but may not appropriate for complex objects

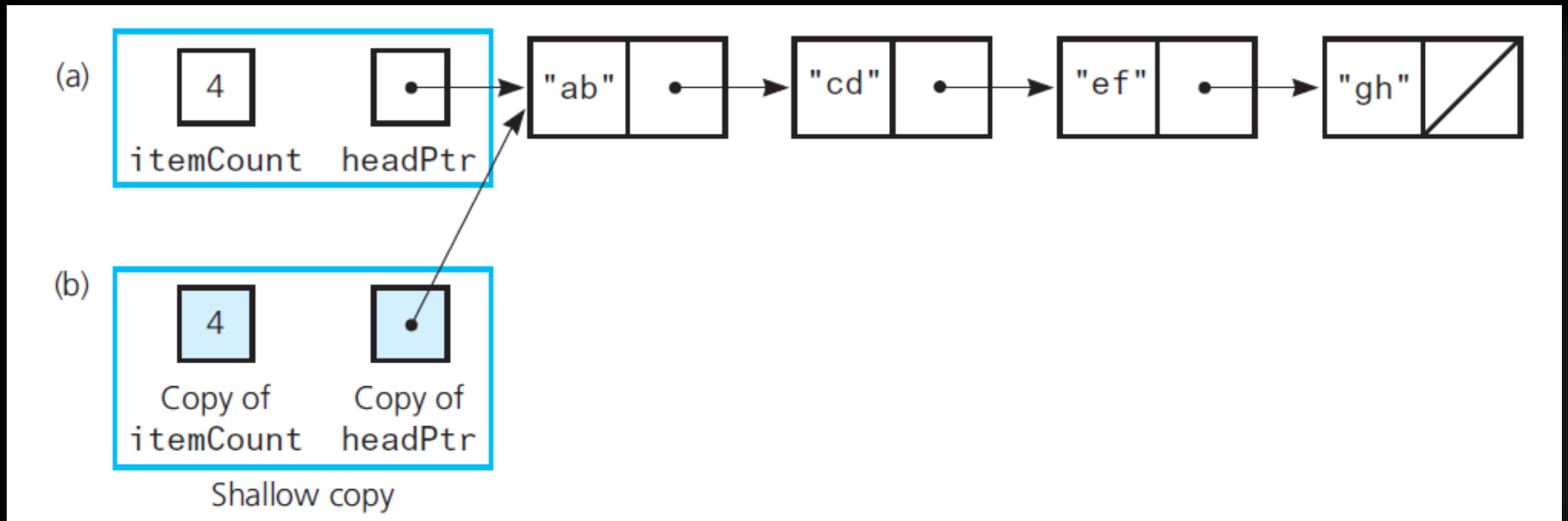
2. Copy an object to **pass by value** as an argument to a function

```
void MyFunction(MyClass arg) {
        /* ... */
}
```

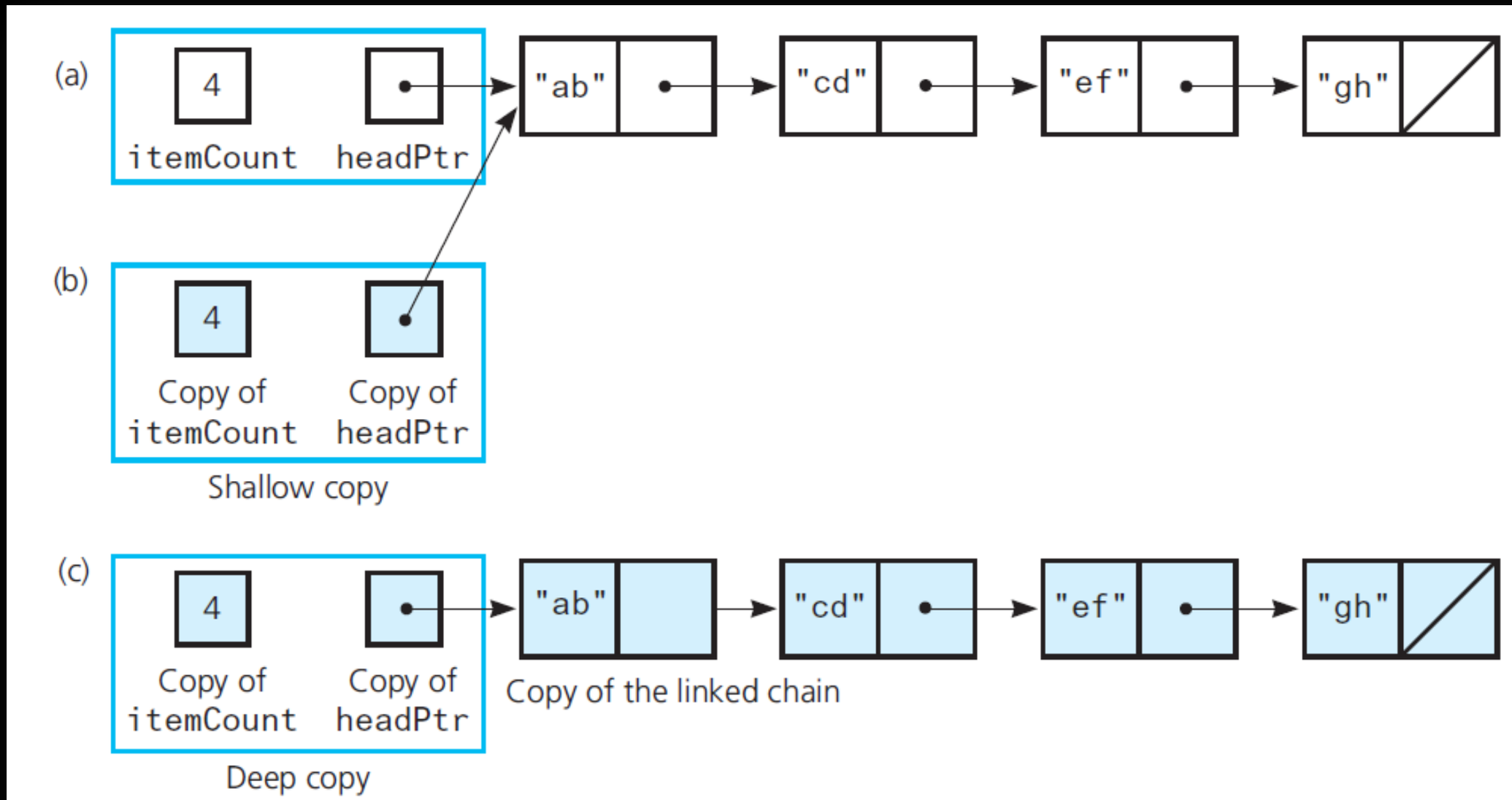
3. Copy an object to be **returned** by a function

```
MyClass MyFunction() {
   MyClass mc;
   return mc;
}
```

Deep vs Shallow Copy



Deep vs Shallow Copy



Overloaded operator=

```
MyClass one;
//Stuff here
MyClass two = one;
```

Instantiation: copy constructor is called

IS DIFFERENT FROM

```
MyClass one, two;
//Stuff here
two = one;
```

Assignment, NOT instantiation: no constructor is called, must overload `operator=` to avoid shallow copy

Different functions/call same implementation

Class must explicitly define deep copy behavior when memory is dynamically allocated

LinkedBag Implementation

```
#include "LinkedBag.hpp"
template<class T>
LinkedBag<T>::LinkedBag(const LinkedBag<T>& a_bag)
{
   item_count_ = a_bag.item_count_;
   Node<T>* orig_chain_ptr = a_bag.head_ptr_;  // Points to nodes in original chain
   if (orig_chain_ptr == nullptr)
      head_ptr_ = nullptr;  // Original bag is empty
   else
   {
      // Copy first node
      head_ptr_ = new Node<T>();
      head_ptr_->setItem(orig_chain_ptr->getItem());

      // Copy remaining nodes
      Node<T>* new_chain_ptr = head_ptr_;      // Points to last node in new chain
      orig_chain_ptr = orig_chain_ptr->getNext();     // Advance original-chain pointe
      while (orig_chain_ptr != nullptr)
      {
         // Get next item from original chain
         T next_item = orig_chain_ptr->getItem();
         // Create a new node containing the next item
         Node<T>* new_node_ptr = new Node<T>(next_item);
         // Link new node to end of new chain
         new_chain_ptr->setNext(new_node_ptr);

         // Advance pointer to new last node
         new_chain_ptr = new_chain_ptr->getNext();
         // Advance original-chain pointer
         orig_chain_ptr = orig_chain_ptr->getNext();
      }  // end while
      new_chain_ptr->setNext(nullptr);  // Flag end of chain
   }  // end if
}  // end copy constructor
```

The `copy constructor`
A constructor whose parameter is an object of the same class

Called when object is initialized with a copy of another object, e.g.
`LinkedBag<string> my_bag = your_bag;`

Copy first node

Two traversing pointers
One to new chain, one to original chain

`while`

Copy item from current node

Create new node with item

Connect new node to new chain

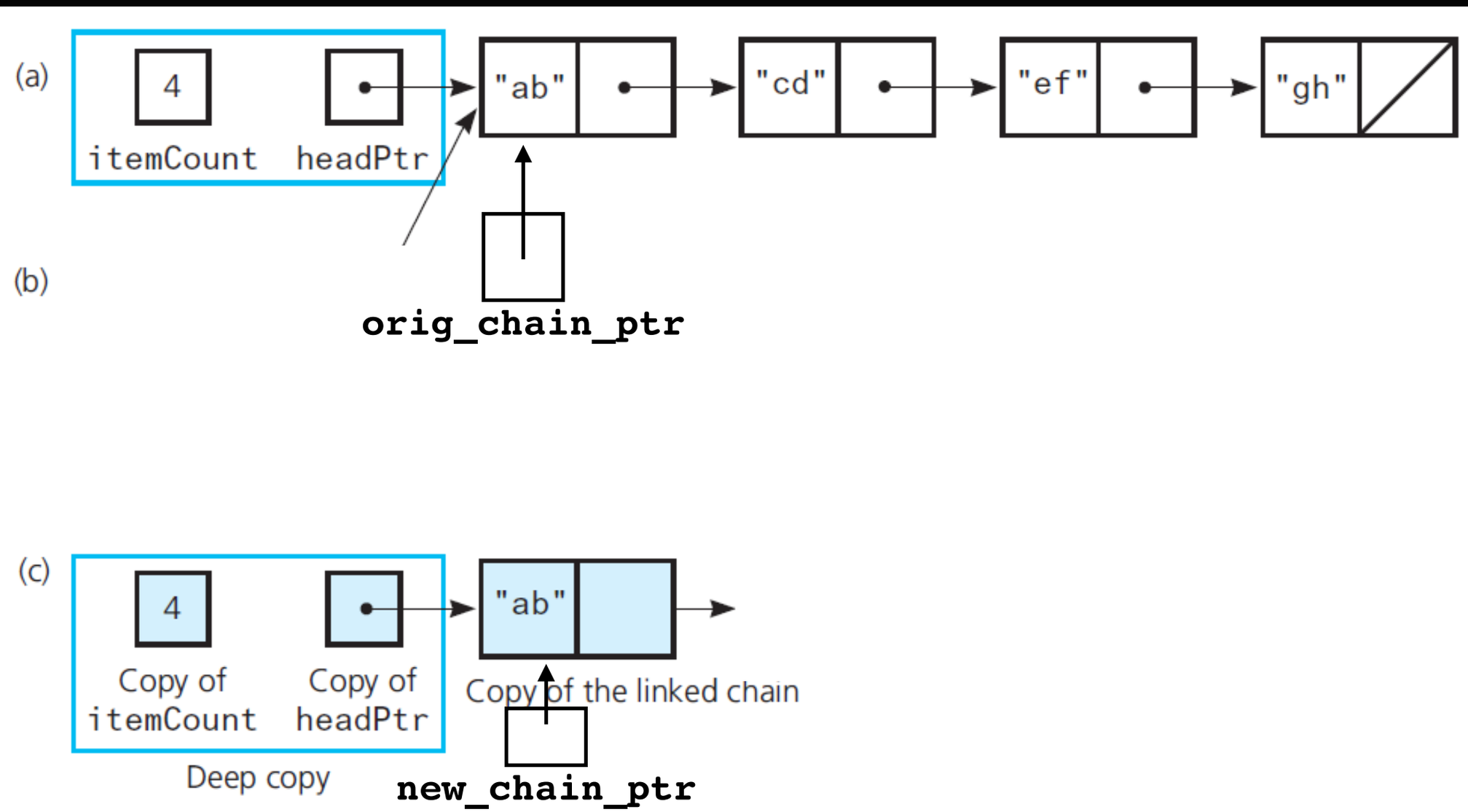
Advance pointer traversing new chain

Advance pointer traversing original chain

Signal last node

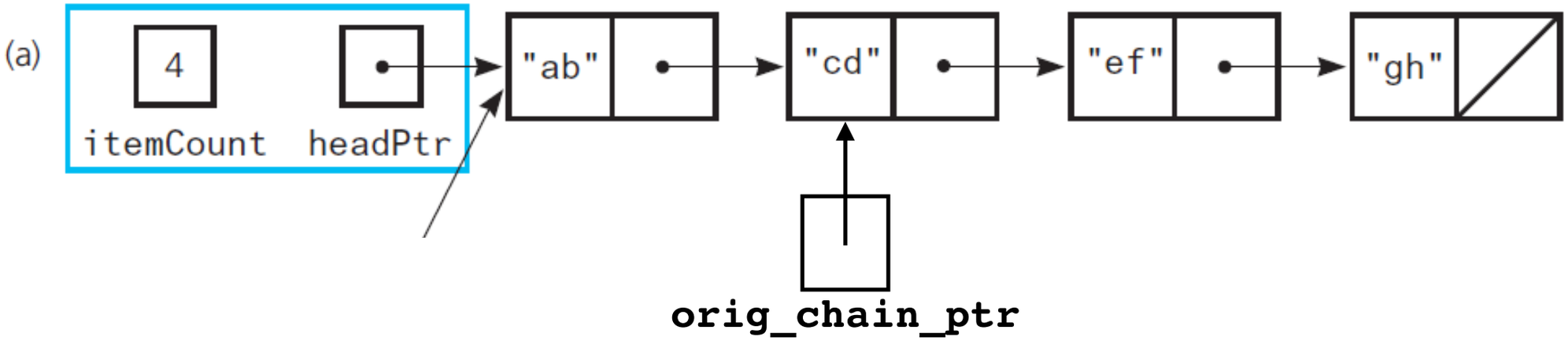
Deep vs Shallow Copy

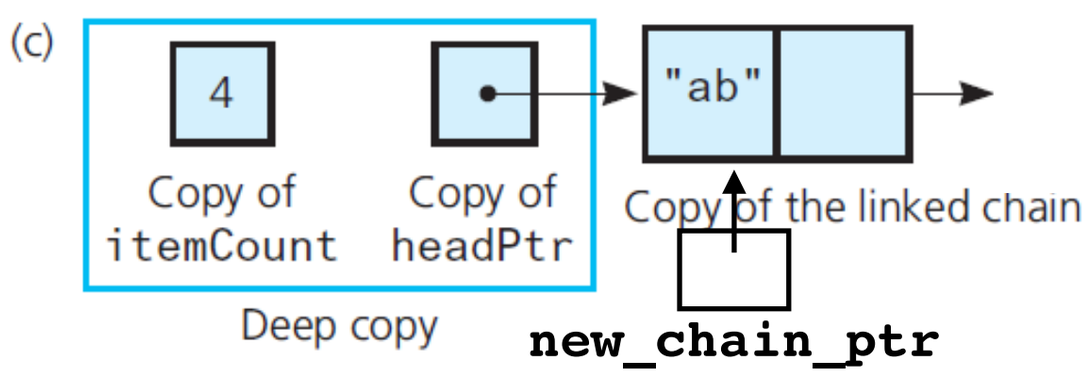
```
// Copy first node
      head_ptr_ = new Node<T>();
      head_ptr_->setItem(orig_chain_ptr->getItem());


      // Copy remaining nodes
      Node<T>* new_chain_ptr = head_ptr_;       //
Points to last node in new chain
      orig_chain_ptr = orig_chain_ptr->getNext();
```



Deep vs Shallow Copy

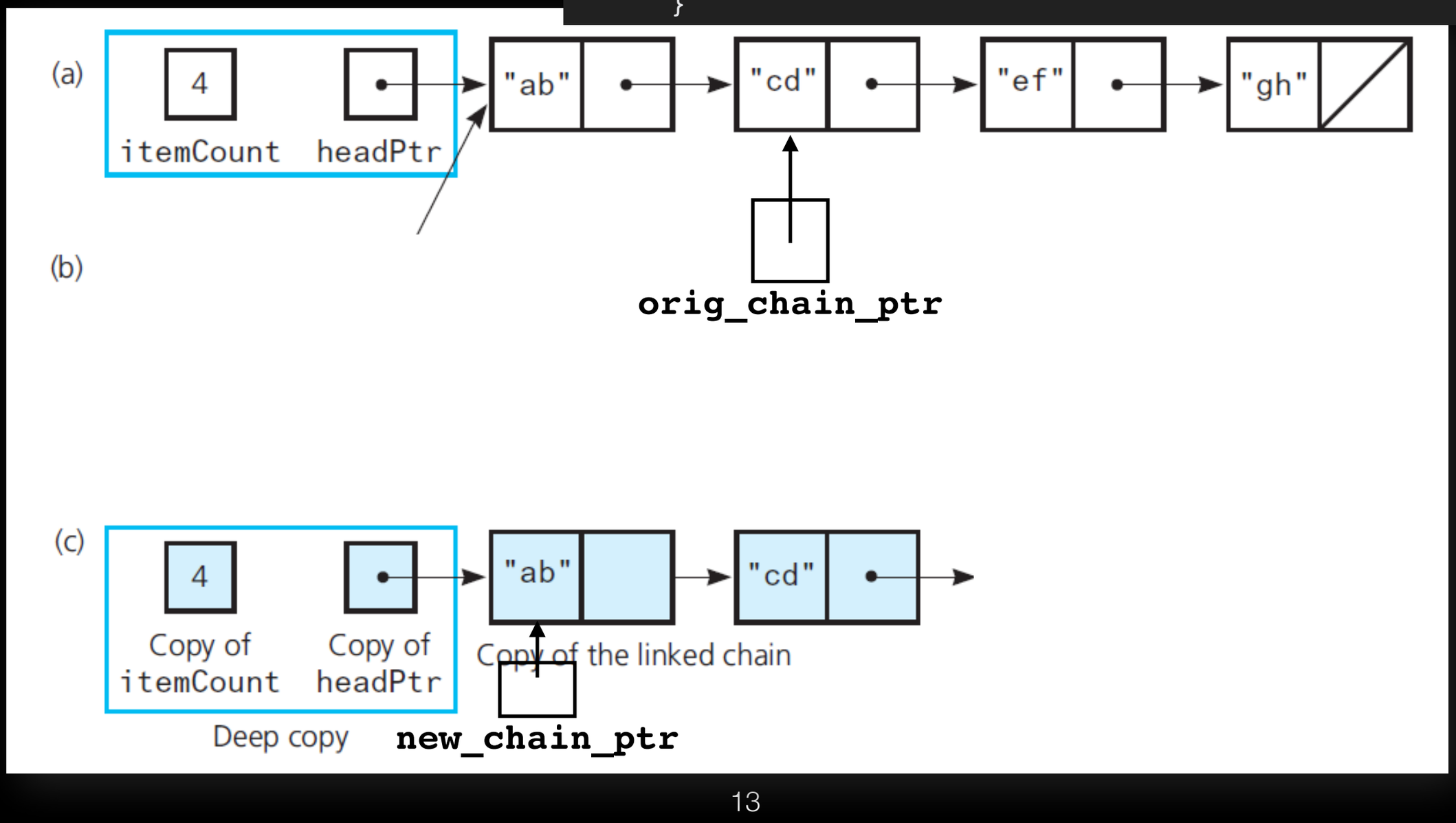
```
// Copy first node
      head_ptr_ = new Node<T>();
      head_ptr_->setItem(orig_chain_ptr->getItem());


      // Copy remaining nodes
      Node<T>* new_chain_ptr = head_ptr_;       //
Points to last node in new chain
      orig_chain_ptr = orig_chain_ptr->getNext();
```





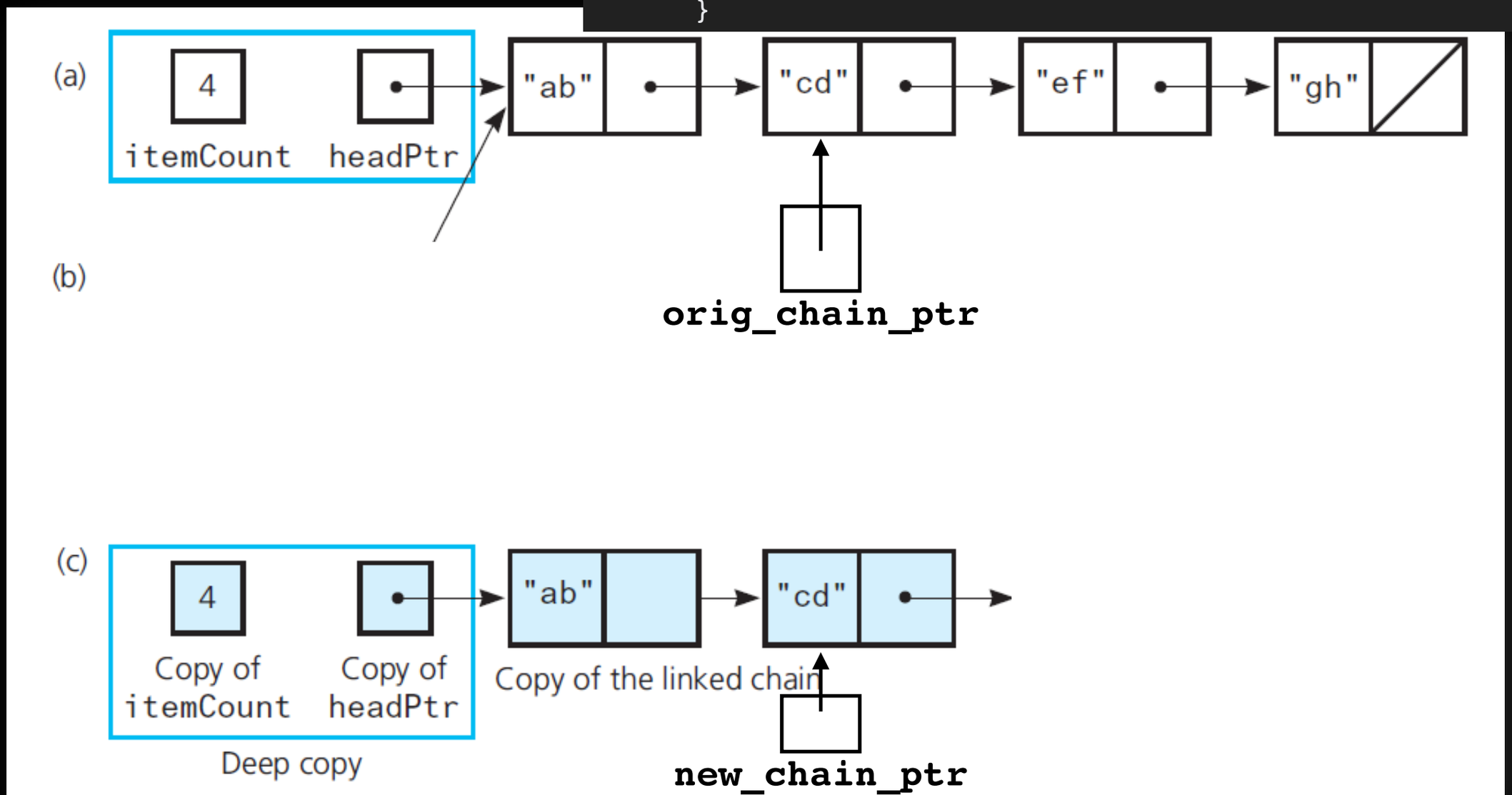
Deep vs Shallow Copy

```
while (orig_chain_ptr != nullptr)
      {
         // Get next item from original chain
         T next_item = orig_chain_ptr->getItem();
         // Create a new node containing the next item
         Node<T>* new_node_ptr = new Node<T>(next_item);
         // Link new node to end of new chain
         new_chain_ptr->setNext(new_node_ptr);
        // Advance pointer to new last node
         new_chain_ptr = new_chain_ptr->getNext();
         // Advance original-chain pointer
         orig_chain_ptr = orig_chain_ptr->getNext();
      }
```



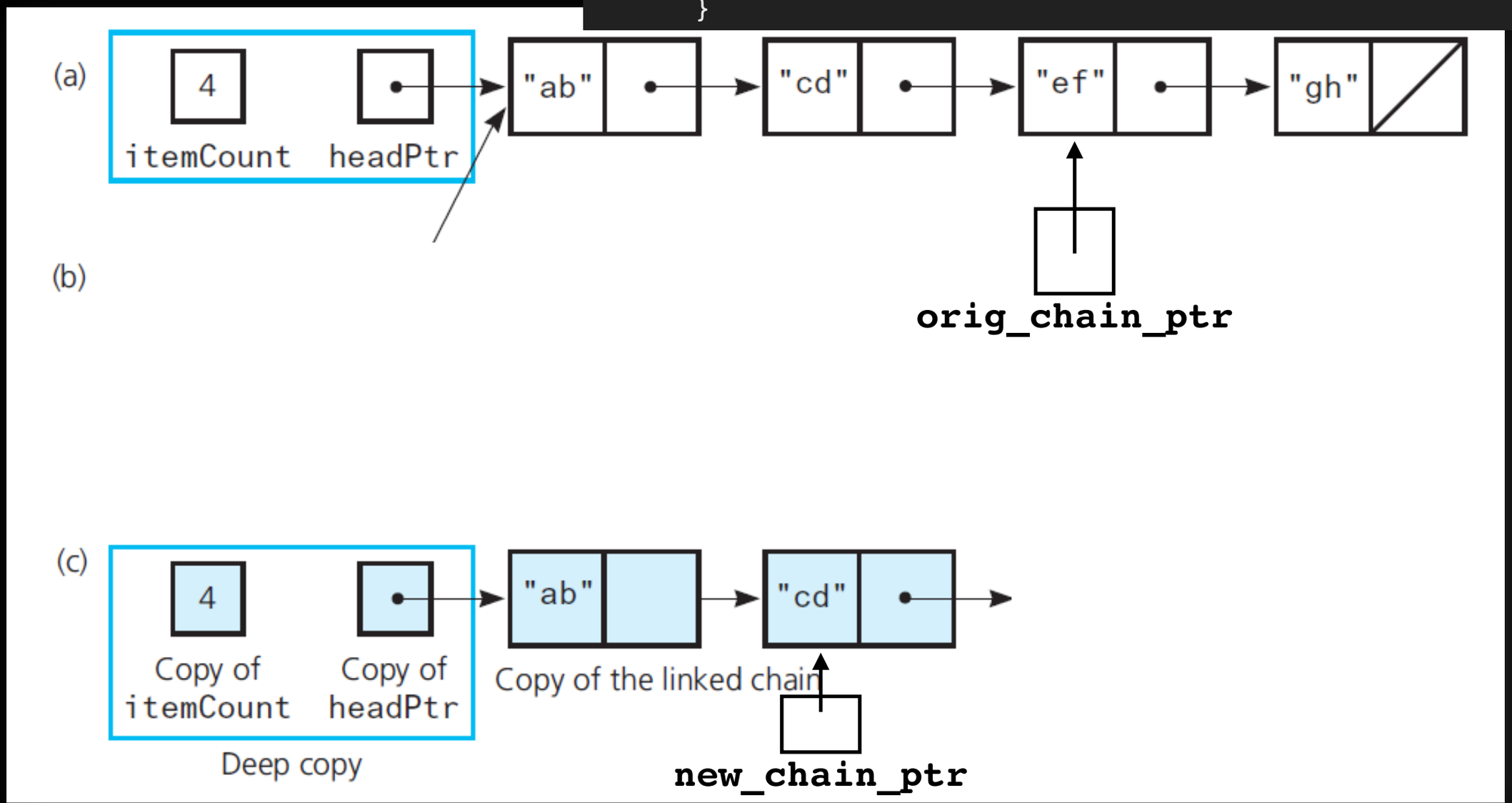
Deep vs Shallow Copy

```
while (orig_chain_ptr != nullptr)
      {
         // Get next item from original chain
         T next_item = orig_chain_ptr->getItem();
         // Create a new node containing the next item
         Node<T>* new_node_ptr = new Node<T>(next_item);
         // Link new node to end of new chain
         new_chain_ptr->setNext(new_node_ptr);
        // Advance pointer to new last node
         new_chain_ptr = new_chain_ptr->getNext();
         // Advance original-chain pointer
         orig_chain_ptr = orig_chain_ptr->getNext();
      }
```



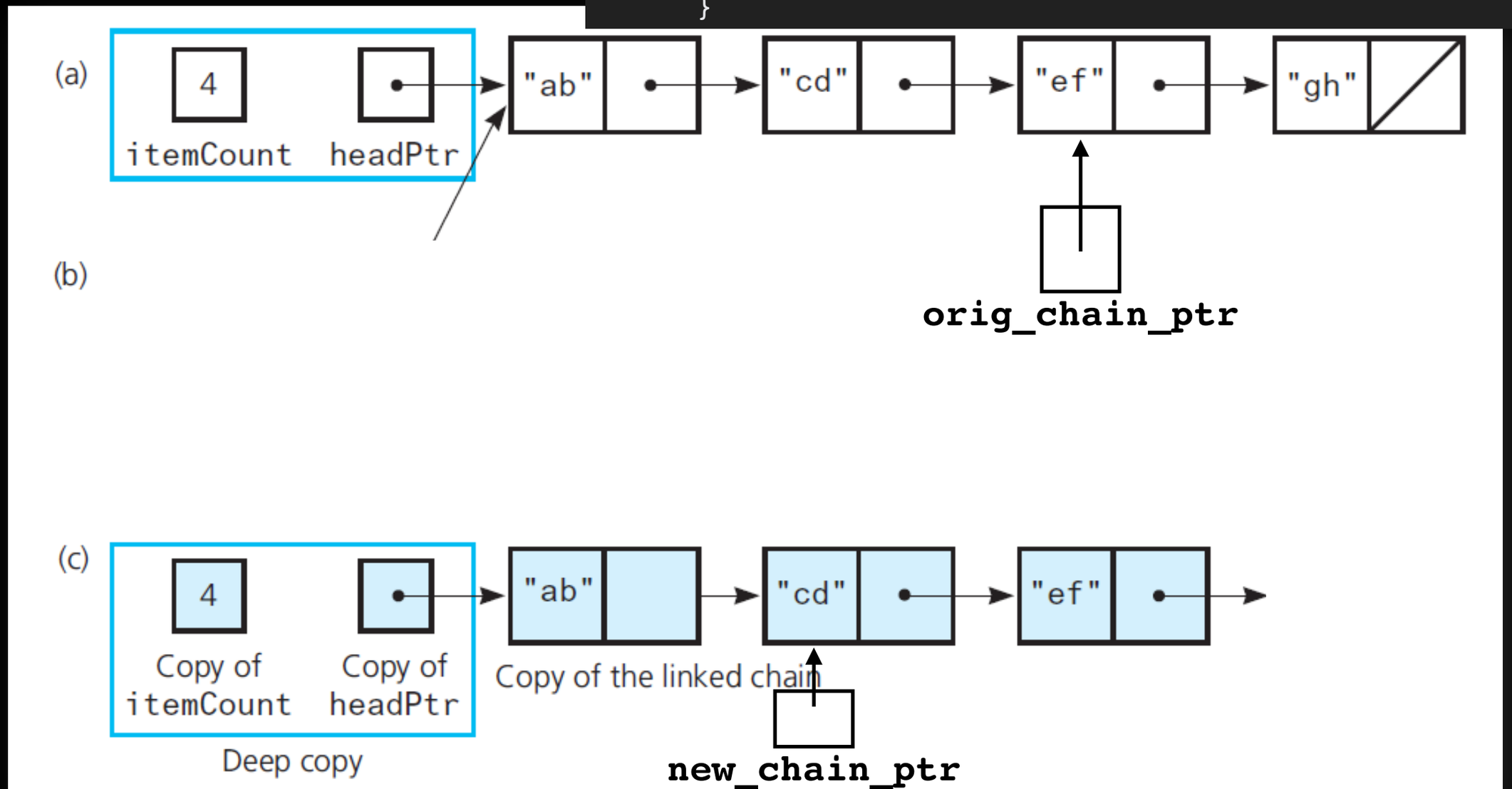
Deep vs Shallow Copy

```
while (orig_chain_ptr != nullptr)
      {
         // Get next item from original chain
         T next_item = orig_chain_ptr->getItem();
         // Create a new node containing the next item
         Node<T>* new_node_ptr = new Node<T>(next_item);
         // Link new node to end of new chain
         new_chain_ptr->setNext(new_node_ptr);
        // Advance pointer to new last node
         new_chain_ptr = new_chain_ptr->getNext();
         // Advance original-chain pointer
         orig_chain_ptr = orig_chain_ptr->getNext();
      }
```



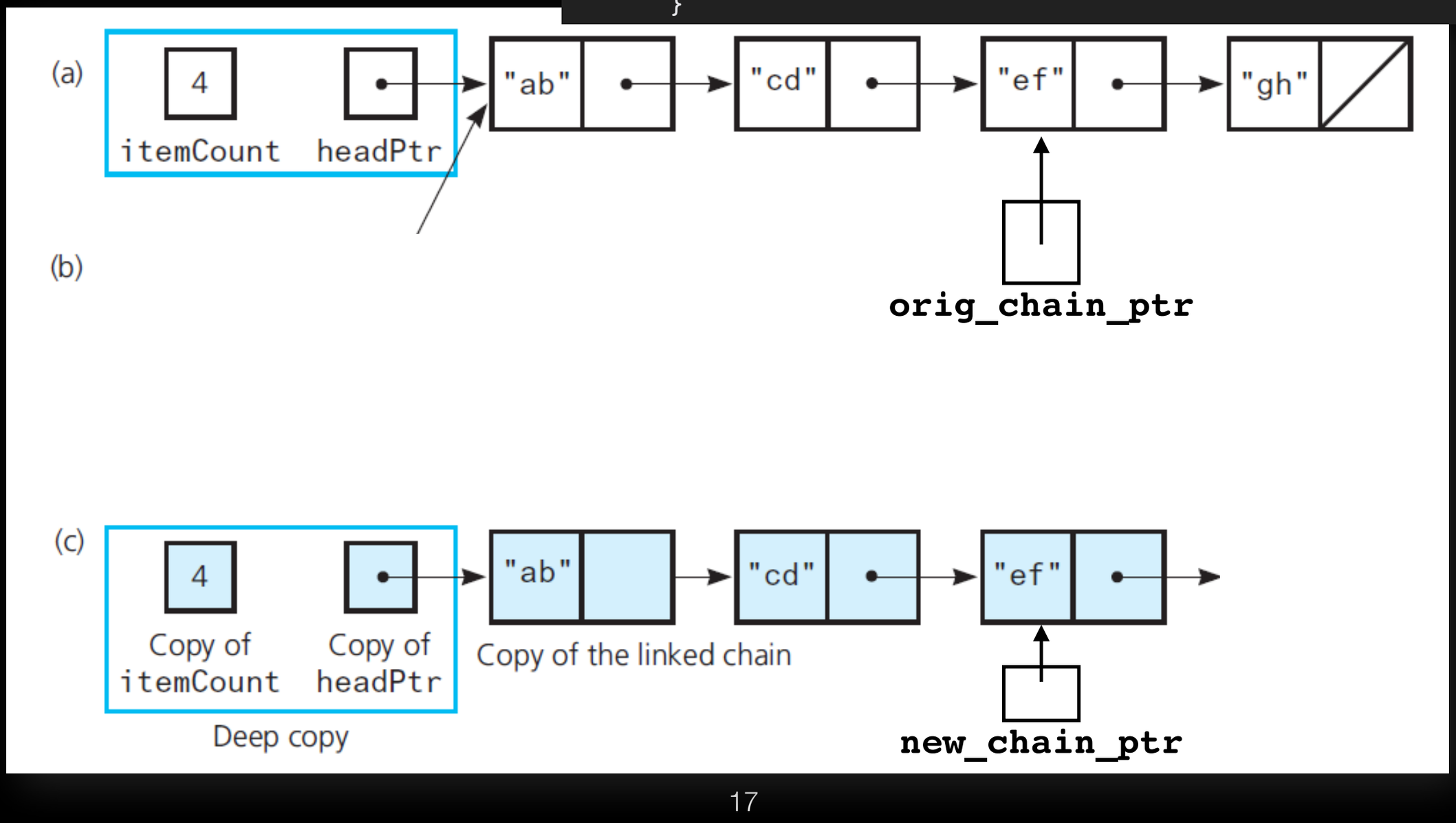
Deep vs Shallow Copy

```
while (orig_chain_ptr != nullptr)
      {
         // Get next item from original chain
         T next_item = orig_chain_ptr->getItem();
         // Create a new node containing the next item
         Node<T>* new_node_ptr = new Node<T>(next_item);
         // Link new node to end of new chain
         new_chain_ptr->setNext(new_node_ptr);
        // Advance pointer to new last node
         new_chain_ptr = new_chain_ptr->getNext();
         // Advance original-chain pointer
         orig_chain_ptr = orig_chain_ptr->getNext();
      }
```



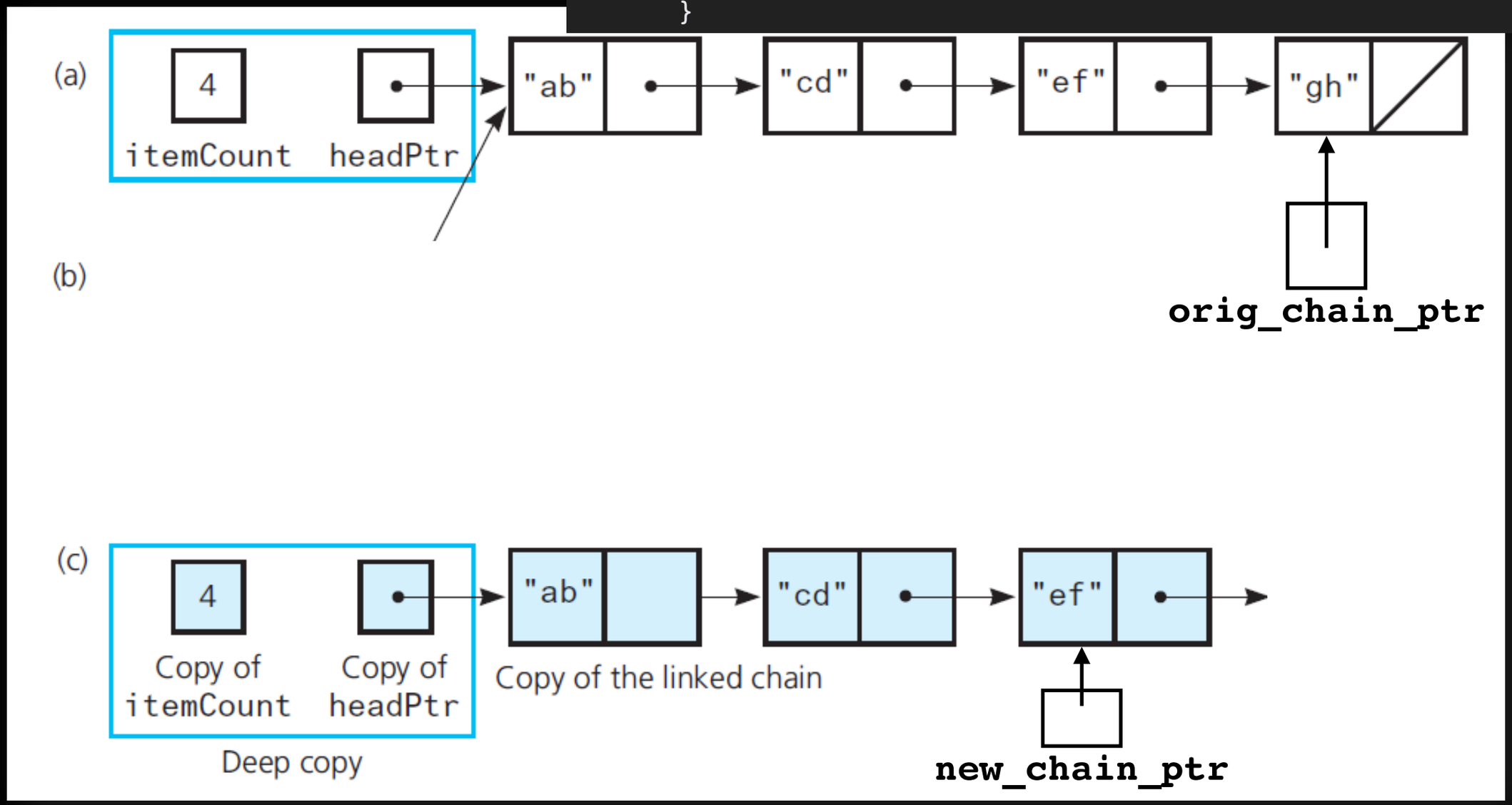
Deep vs Shallow Copy

```
while (orig_chain_ptr != nullptr)
      {
         // Get next item from original chain
         T next_item = orig_chain_ptr->getItem();
         // Create a new node containing the next item
         Node<T>* new_node_ptr = new Node<T>(next_item);
         // Link new node to end of new chain
         new_chain_ptr->setNext(new_node_ptr);
        // Advance pointer to new last node
         new_chain_ptr = new_chain_ptr->getNext();
         // Advance original-chain pointer
         orig_chain_ptr = orig_chain_ptr->getNext();
      }
```



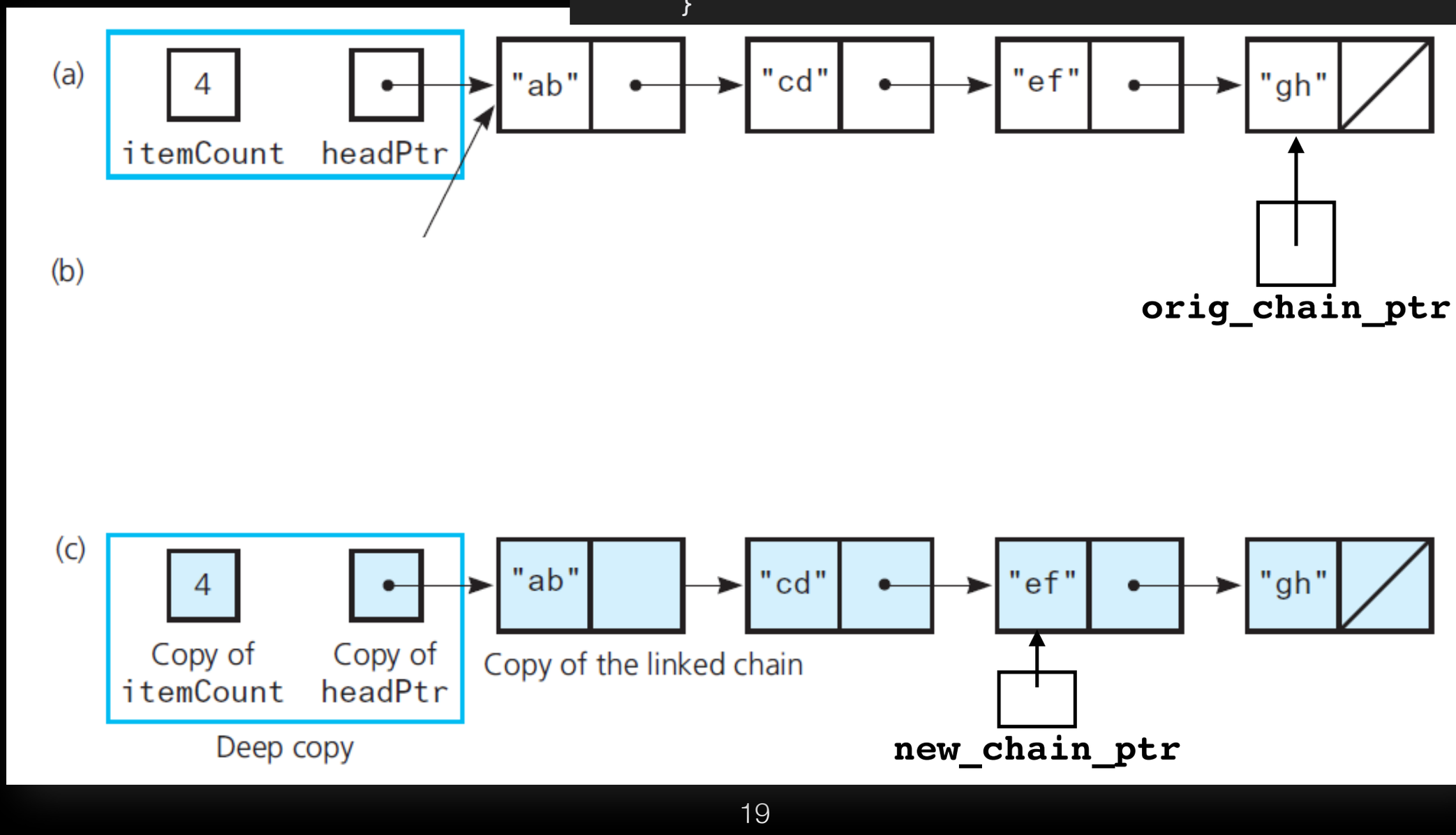
Deep vs Shallow Copy

```
while (orig_chain_ptr != nullptr)
      {
         // Get next item from original chain
         T next_item = orig_chain_ptr->getItem();
         // Create a new node containing the next item
         Node<T>* new_node_ptr = new Node<T>(next_item);
         // Link new node to end of new chain
         new_chain_ptr->setNext(new_node_ptr);
        // Advance pointer to new last node
         new_chain_ptr = new_chain_ptr->getNext();
         // Advance original-chain pointer
         orig_chain_ptr = orig_chain_ptr->getNext();
      }
```



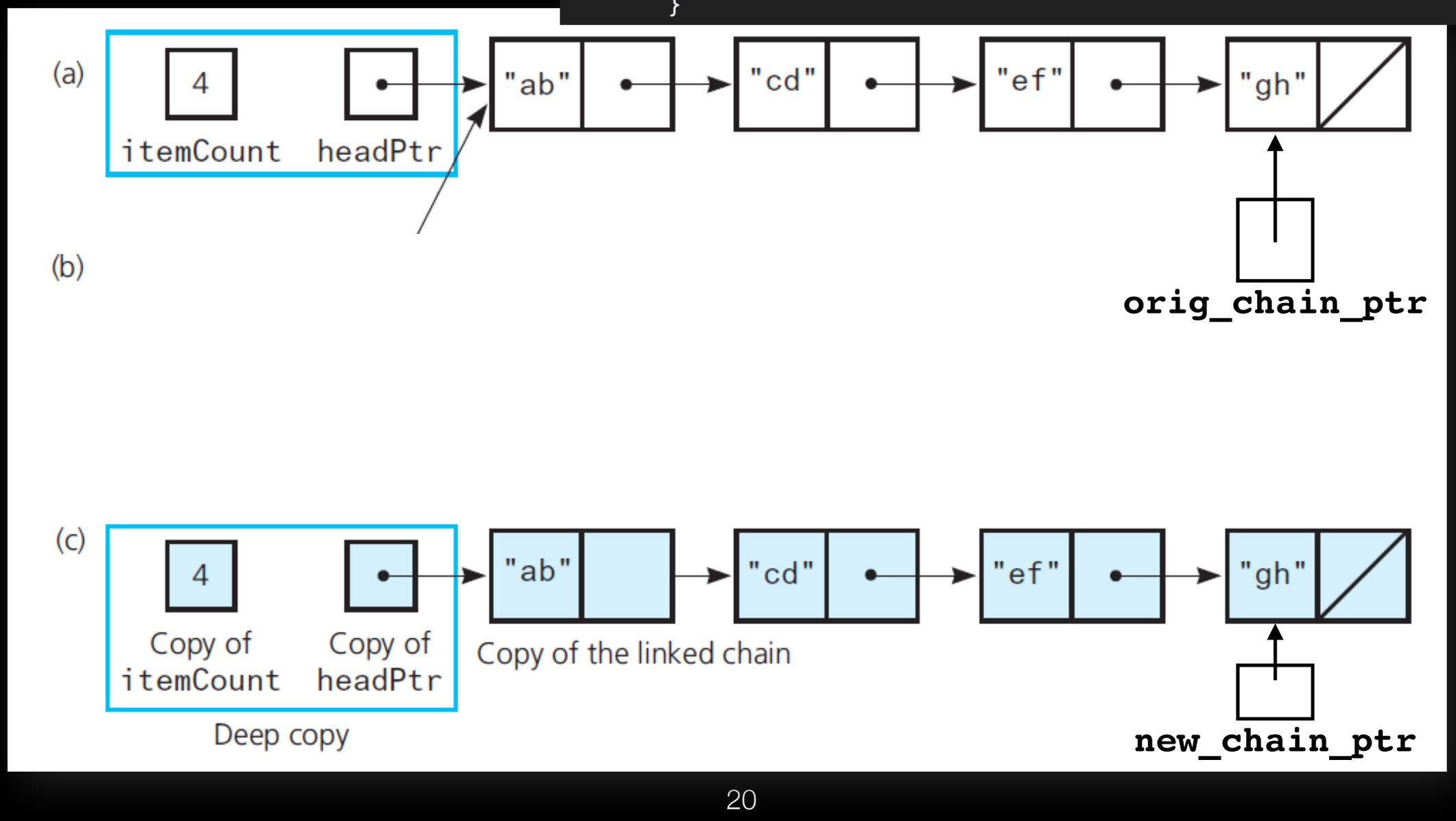
Deep vs Shallow Copy

```
while (orig_chain_ptr != nullptr)
      {
         // Get next item from original chain
         T next_item = orig_chain_ptr->getItem();
         // Create a new node containing the next item
         Node<T>* new_node_ptr = new Node<T>(next_item);
         // Link new node to end of new chain
         new_chain_ptr->setNext(new_node_ptr);
        // Advance pointer to new last node
         new_chain_ptr = new_chain_ptr->getNext();
         // Advance original-chain pointer
         orig_chain_ptr = orig_chain_ptr->getNext();
      }
```



Deep vs Shallow Copy

```
while (orig_chain_ptr != nullptr)
      {
         // Get next item from original chain
         T next_item = orig_chain_ptr->getItem();
         // Create a new node containing the next item
         Node<T>* new_node_ptr = new Node<T>(next_item);
         // Link new node to end of new chain
         new_chain_ptr->setNext(new_node_ptr);
        // Advance pointer to new last node
         new_chain_ptr = new_chain_ptr->getNext();
         // Advance original-chain pointer
         orig_chain_ptr = orig_chain_ptr->getNext();
      }
```



Efficiency Considerations

Every time you pass or return an object by value:

- Call copy constructor
- Call destructor

O(n)

For linked chain: **O(n)**

- Traverse entire chain to copy (`n "steps"`)
- Traverse entire chain to destroy (`n "steps"`)

Preferred call by const reference:

```
myFunction(const MyClass& object);
```

Move Semantics

Copying can be time/space consuming, especially if large amount of data

Copying often involves making copy and destroying original (e.g, pass by value, return by value, old-school swap with intermediate): inefficient

More efficient to **transfer ownership** of resources to another object

lvalues and *rvalues*

lvalue = rvalue

Examples:

```
int x = 2;
int y = x+1;
x = y;
x = y + z;
string msg = “hello”;
bool pass = computeGrades(student);
```

The return value, not the function

Lvalues can be referred to by name, pointer or lvalue reference: i.e. they have an address

Rvalues are literals or temporary objects that are the result of evaluating expressions, or are copied into or returned by functions, they don’t have an address and cannot have a value assigned to it

Can have lvalue and rvalue of same type.

Move Semantics

Rvalues are eligible for **move operations**

After object x is moved into object y:

- y is equivalent to the former value of x
- x is in a special state called the *moved-from state*
- Object in moved-from state can only be reassigned or destructed (becomes an rvalue)
- rvalue is semantically temporary, thus it is more likely to be put in temporary memory or optimized

`std::move()`

A type-cast

Converts an lvalue to an rvalue

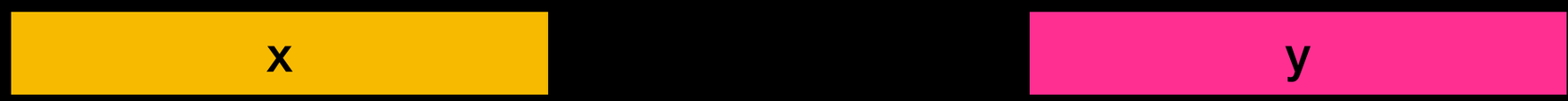
Allows the **efficient transfer of resources** from the moved object to another

```
y =   std::move(x);
```

x is now treated as an rvalue

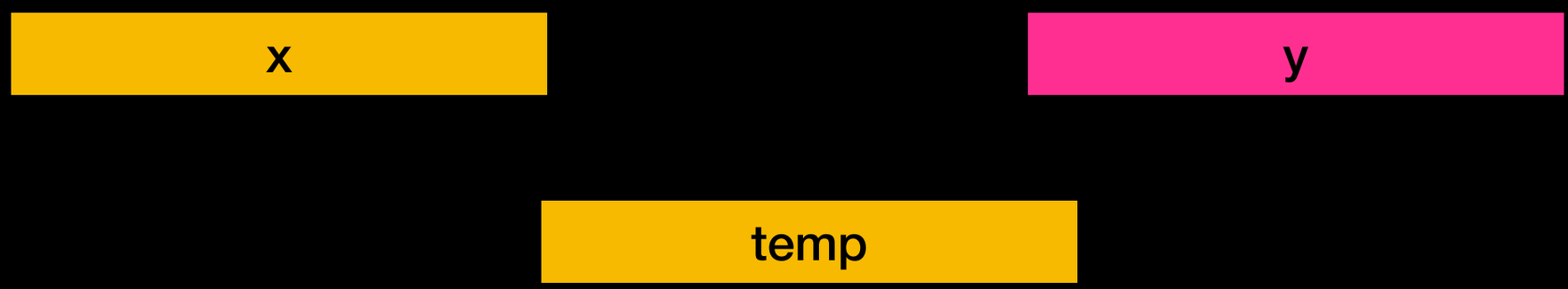
Old-school swap

```
void swap(vector<string>& x, vector<string>& y)
{
        vector<string> temp{x};
        x = y;
        y = temp;
}
```



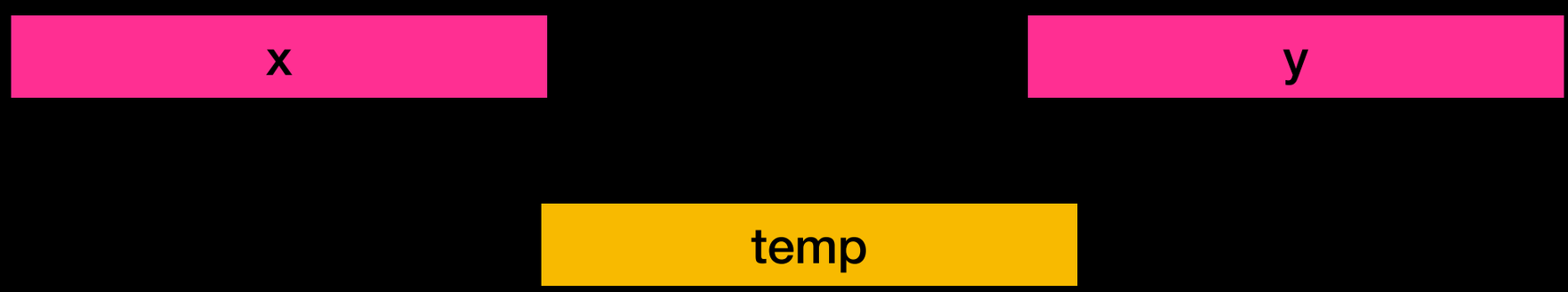
Old-school swap

```
void swap(vector<string>& x, vector<string>& y)
{
        vector<string> temp{x};
        x = y;
        y = temp;
}
```



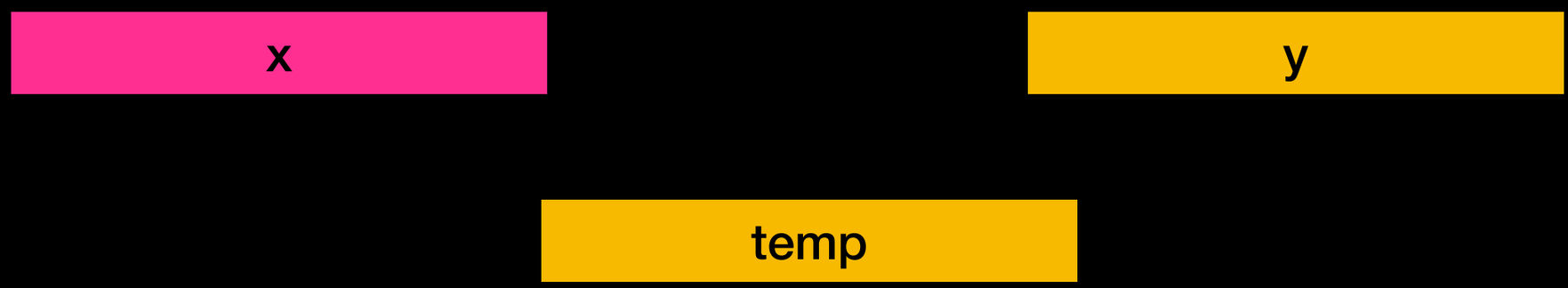
Old-school swap

```
void swap(vector<string>& x, vector<string>& y)
{
        vector<string> temp{x};
        x = y;
        y = temp;
}
```



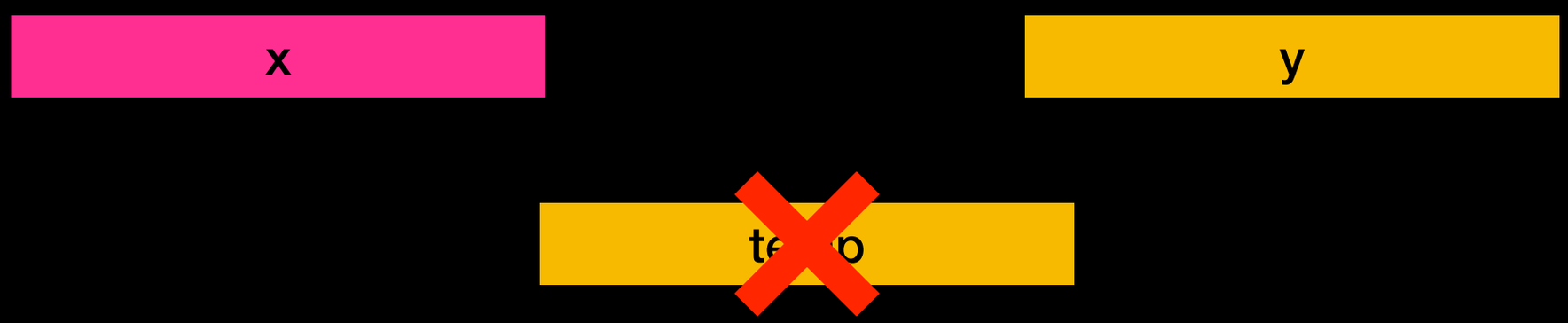
Old-school swap

```
void swap(vector<string>& x, vector<string>& y)
{
        vector<string> temp{x};
        x = y;
        y = temp;
}
```



Old-school swap

```
void swap(vector<string>& x, vector<string>& y)
{
        vector<string> temp{x};
        x = y;
        y = temp;
}
```

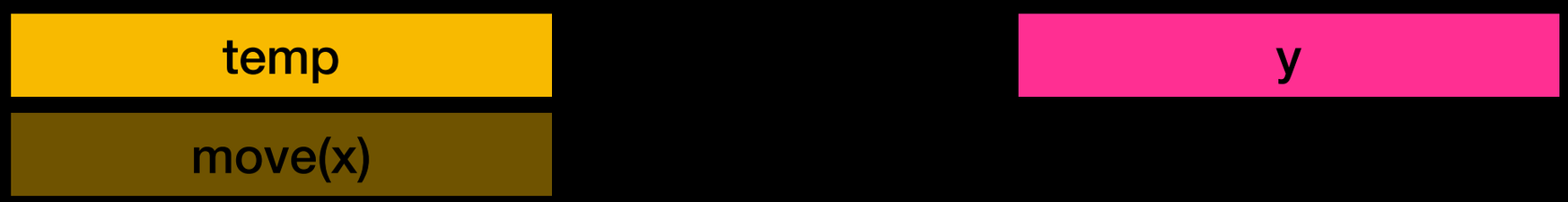


std::swap

```
void swap(vector<string>& x, vector<string>& y)
{
        vector<string> temp{std::move(x)};
        x = std::move(y);
        y = std::move(temp);
}
```

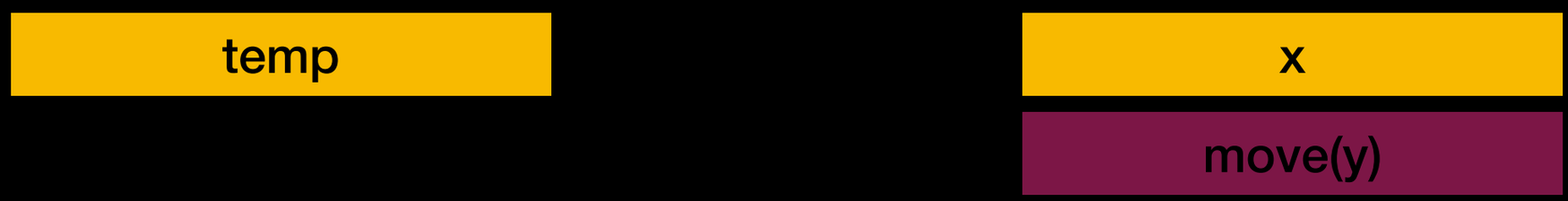
std::swap

```
void swap(vector<string>& x, vector<string>& y)
{
        vector<string> temp{std::move(x)};
        x = std::move(y);
        y = std::move(temp);
}
```



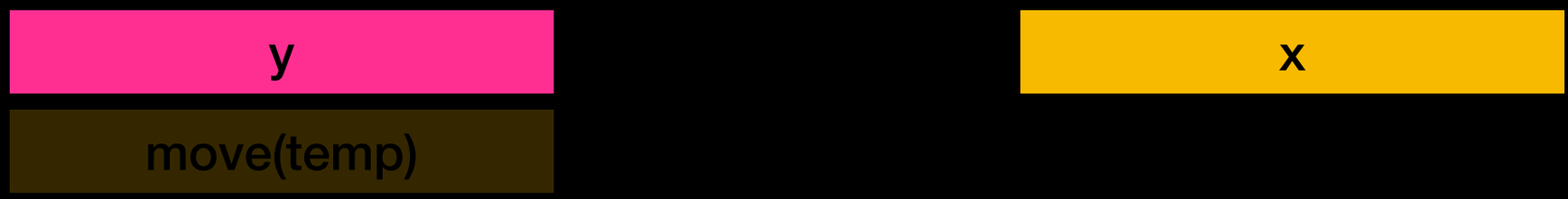
std::swap

```
void swap(vector<string>& x, vector<string>& y)
{
        vector<string> temp{std::move(x)};
        x = std::move(y);
        y = std::move(temp);
}
```



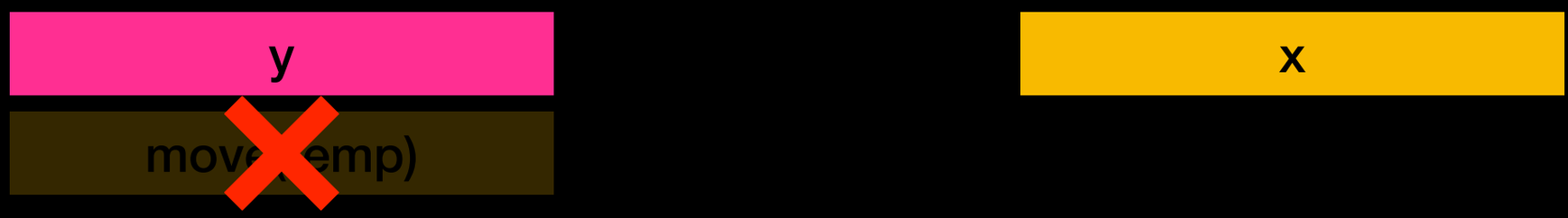
std::swap

```
void swap(vector<string>& x, vector<string>& y)
{
        vector<string> temp{std::move(x)};
        x = std::move(y);
        y = std::move(temp);
}
```



std::swap

```
void swap(vector<string>& x, vector<string>& y)
{
        vector<string> temp{std::move(x)};
        x = std::move(y);
        y = std::move(temp);
}
```



std::swap

Now part of the STL

Can use for any STL type

Example:

```
vector<string> x;
vector<string> y;
//do stuff …

std::swap(x,y);
```

Move Constructor and Assignment

Triggered similarly to copy constructor and assignment operator, but right hand side is rvalue

Explicitly implement move semantics for the class

When defining one should probably define all 5 (Copy constructor, Move constructor, Assignment operator, Move-assignment operator, Destructor)

More in CSci 335

Move Constructor

Implements move semantics instead of copy when:

1. Initialize one object from rvalue

```
MyClass one = rvalue;
```

2. Pass by rvalue reference

rvalue reference

```
void myFunction(MyClass&& arg) {
        /* ... */
}
```

Performs member-wise moves on non-static members of the class

Compiler will NOT generate move constructor if any copy operation is explicitly defined

LinkedBag Implementation

The `move constructor`
A constructor whose parameter is an **rvalue** reference of the same class

```
#include "LinkedBag.hpp"
template<class T>
LinkedBag<T>::LinkedBag(LinkedBag<T>&& a_bag):
item_count_{a_bag.item_count_},
head_ptr_{a_bag.head_ptr_}
{
    a_bag.item_count_ = 0;
    a_bag.head_ptr_ = nullptr;

}  // end move constructor
```

rvalue reference

Move nodes to this bag

No longer points to moved bag

O(1)

```
MyClass one = rvalue;

one = rvalue;
```

LinkedBag Implementation

The `move assignment operator`
std::swap is an O(1) operation, swap with **rvalue** which is about to be destructed by the system anyway

rvalue reference

```
#include "LinkedBag.hpp"
template<class T>
void LinkedBag<T>::operator=(LinkedBag<T>&& rhs)
{
    std::swap(item_count_, rhs.item_count_);
    std::swap(head_ptr_, rhs.head_ptr_);

}   // end move assignment operator
```

Swap bags

O(1)

Exception Handling

Problem With Raw Pointers

```
void func() {
    // Dynamic memory allocation
    Classroom* room = new Classroom(40);
    // An exception is thrown here
    throw std::runtime_error("Failed");
    // Memory leak happens as we lose access
    // to the dynamically allocated classroom
    delete room;    // This line never executes
}
```

- ➔ When a line of code throws an exception, normal execution halts and C++ cleans up local stack variables automatically
- ➔ However, the dynamically allocated memory in the heap is missed resulting in a memory leak
- ➔ C++11 introduces smart pointers that allow automated and safe memory management

Smart Pointers

The following slides are provided by Professor Tiziana Ligorio

Managed Pointers Motivation

What happens when program that dynamically allocated memory relinquishes control in the middle of execution because of an exception?

Managed Pointers Motivation

What happens when program that dynamically allocated memory relinquishes control in the middle of execution because of an exception?

Dynamically allocated memory never released!!!

Managed Pointers Motivation

Whenever using dynamic memory allocation and exception handling together must consider ways to prevent memory leaks

Memory Leak

```
template<class T>
T List<T>::getItem(size_t position) const
{
    Node<T>* pos_ptr = getPointerTo(position);
    if(pos_ptr == nullptr)
        throw(std::out_of_range("getItem called with empty list or invalid position"));
    else
        return pos_ptr->getItem();

}
```

out_of_range exception thrown here

```
T someFunction(const List<T>& some_list)
{
    //code here that dynamically allocates memory
    T an_item;
    //code here
    an_item = some_list.getItem(n);
    // delete dynamically allocated memory

}
```

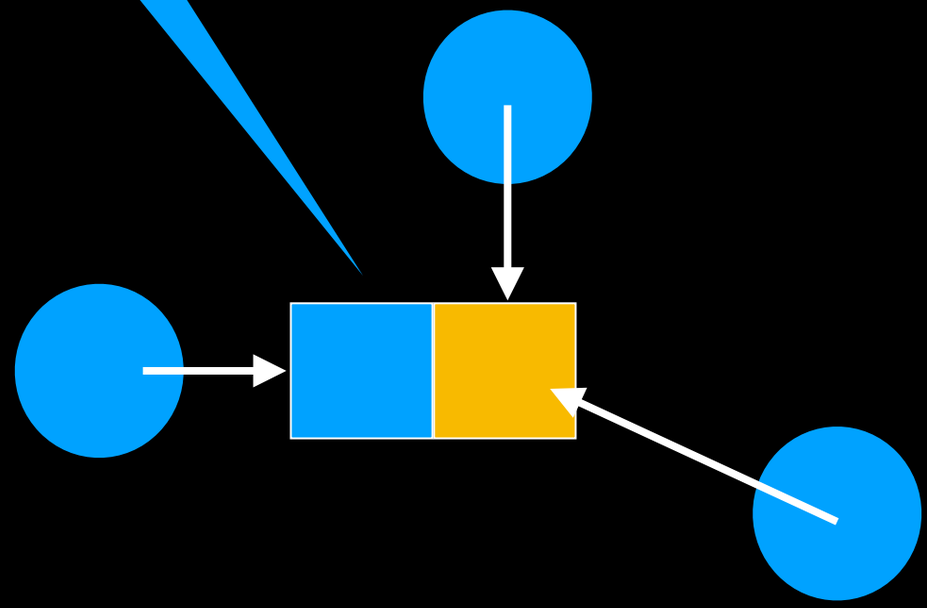
out_of_range exception not handled here

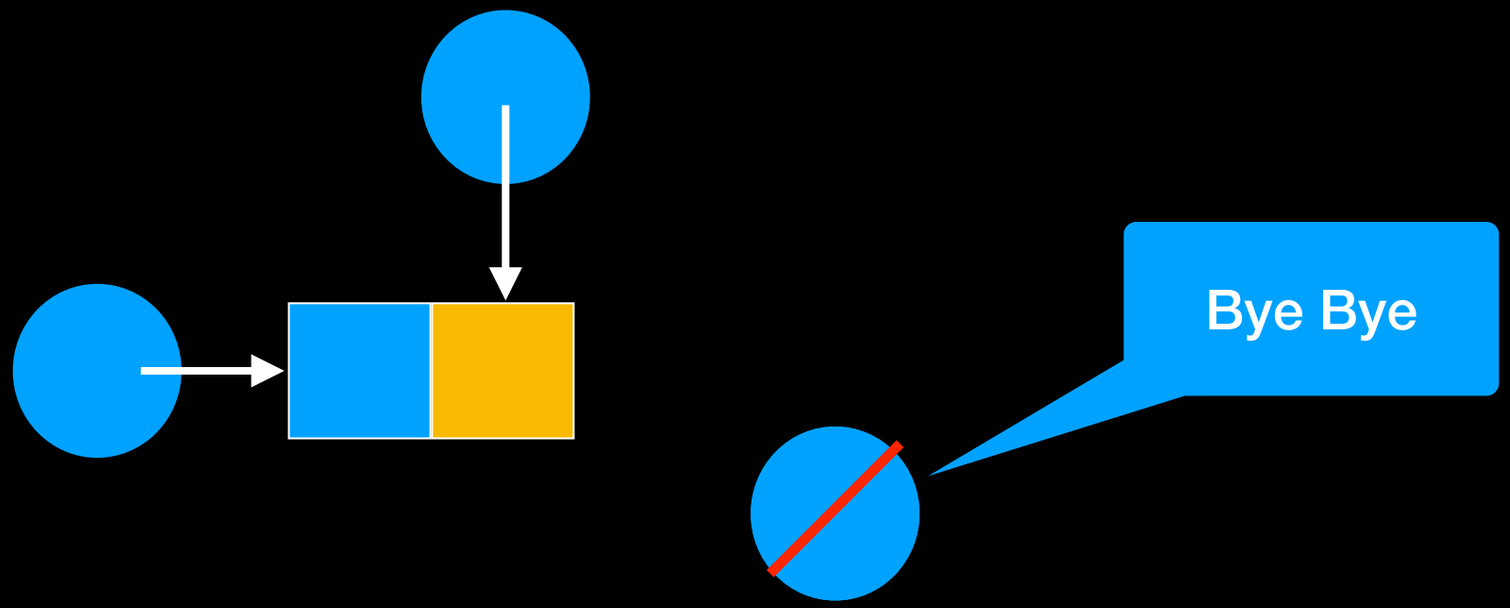
```
int main()
{
    List<string> my_list;
    try
    {
        std::string some_string = someFunction(my_list);
    }
    catch(const std::out_of_range& problem)
    {
        //code to handle exception here
    }
    //more code here
    return 0;
}
```

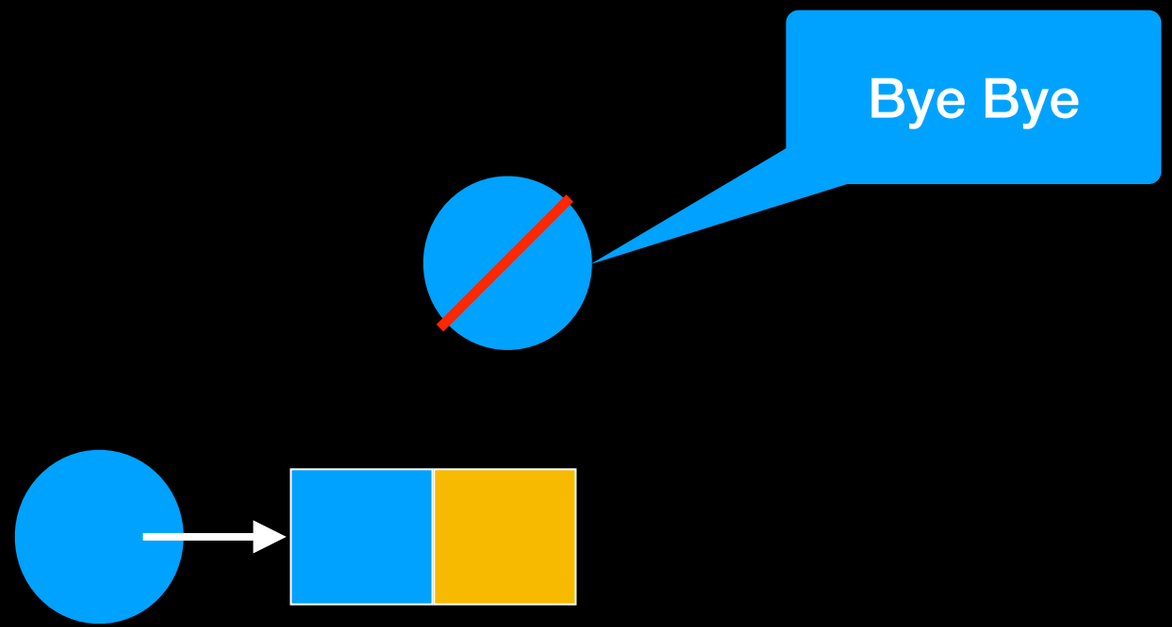
out_of_range exception handled here

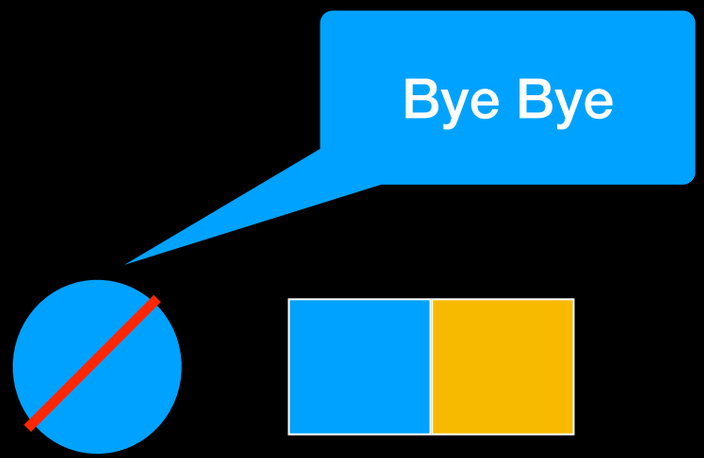
Dynamically allocated object

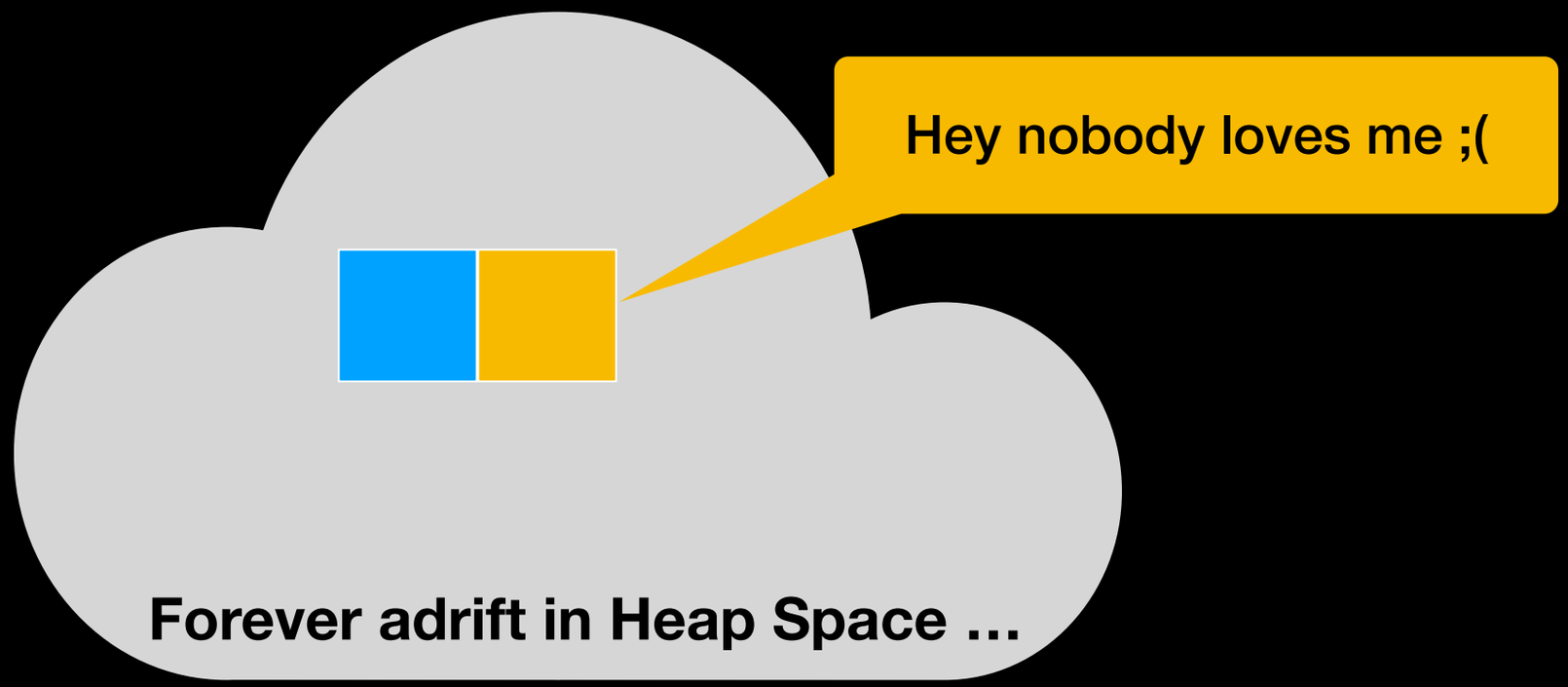
Pointers are not aware of each other

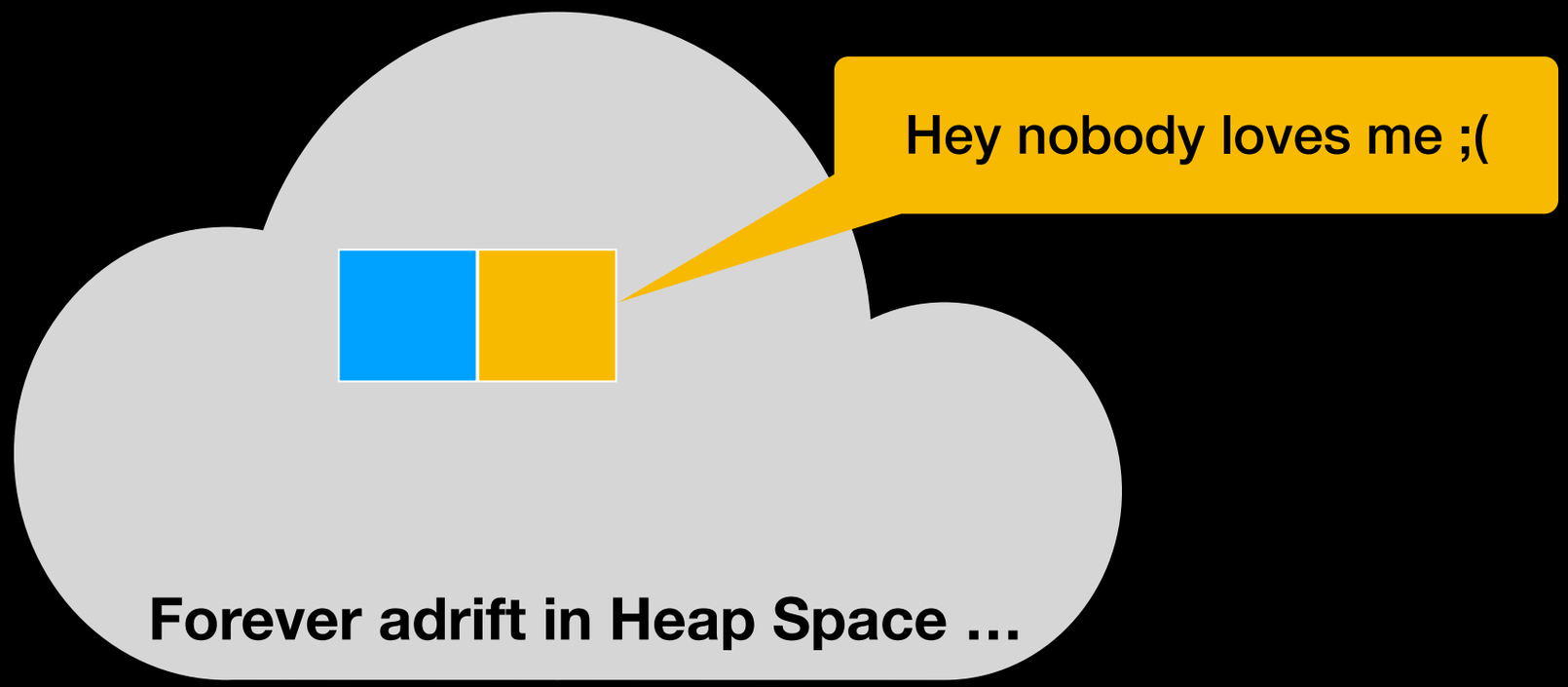



Bye Bye


Bye Bye


Bye Bye


Hey nobody loves me ;(
Forever adrift in Heap Space ...



Programmer responsible for keeping track

Ownership

A pointer is said to **own** a dynamically allocated object if it is responsible for deleting it

If any node is disconnected it is lost on heap

Nodes must be deleted before disconnecting from chain

If multiple pointers point to same node it can be hard to keep track who is responsible for deleting it

Smart/Managed Pointer

A Light Introduction

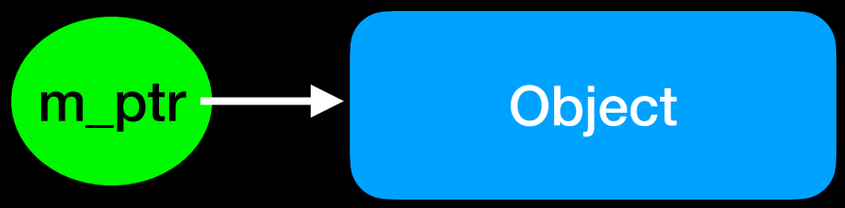
Smart/Managed Pointer

Smart pointer:

- An object
- Acts like a **raw pointer**
-Provides automatic memory management
  (at some performance cost)

Distinguish from"smart"

C++14



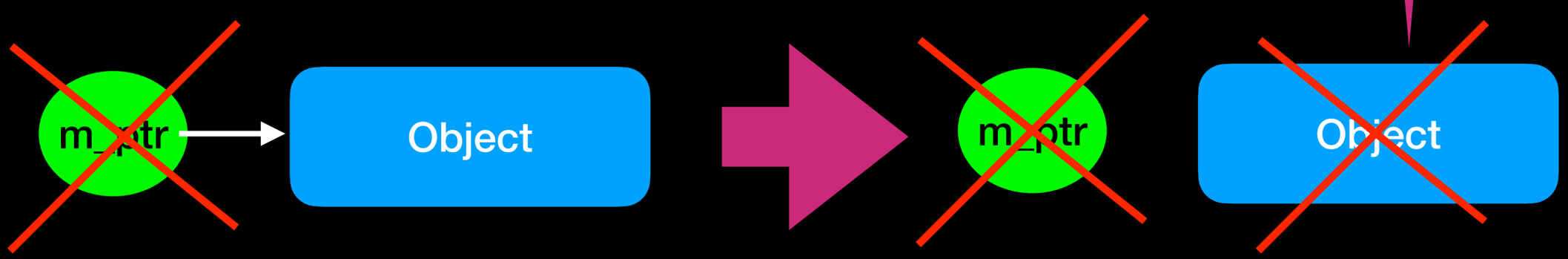
A non-trivial sentence but we will leave it at that

Smart/Managed Pointer

Smart pointer:

- An object
- Acts like a **raw pointer**
-Provides automatic memory management
(at some performance cost)

Smart Pointer destructor automatically invokes destructor of object it points to



Smart/Managed Pointers

Smart pointer ownership = object's destructor automatically invoked when pointer goes out of scope or set to `nullptr`

3 types:

- `shared_ptr`
- `unique_ptr`
- `weak_ptr`

Shared ownership: keeps track of # of pointers to one object. The last one must delete object

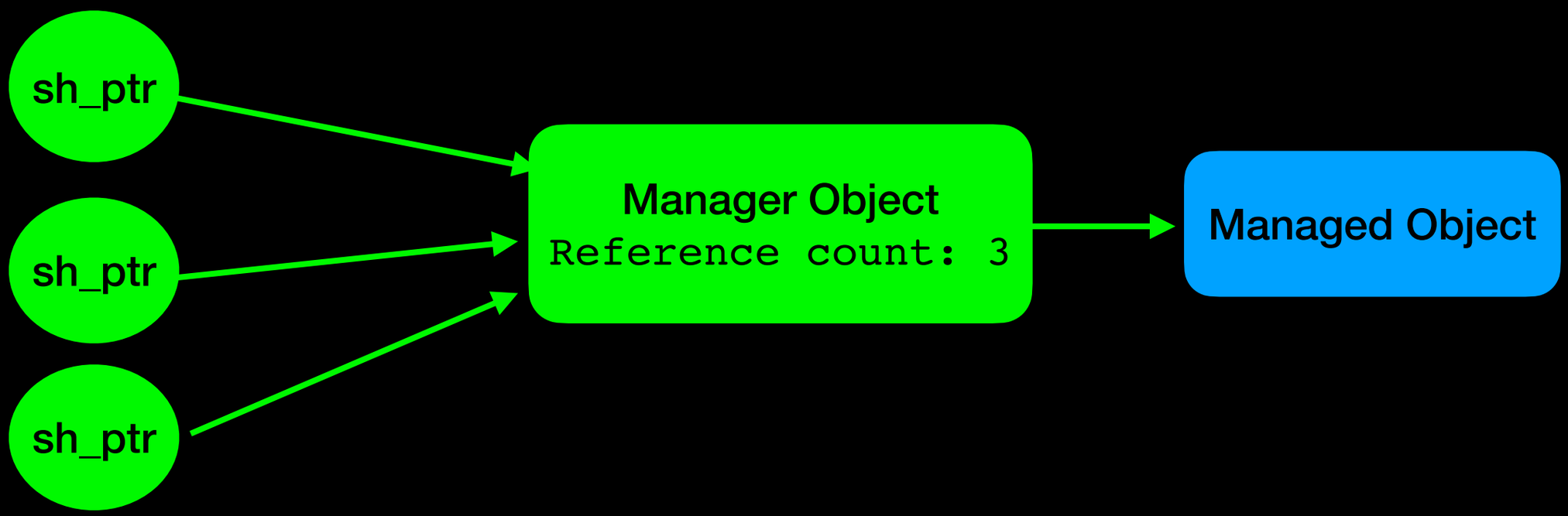
Unique ownership: only smart pointer allowed to point to the object

Points but **does not own**

Smart/Managed Pointers

`shared_ptr`

- keep count how many references to same object
- last pointer responsible for deleting object



Smart/Managed Pointers

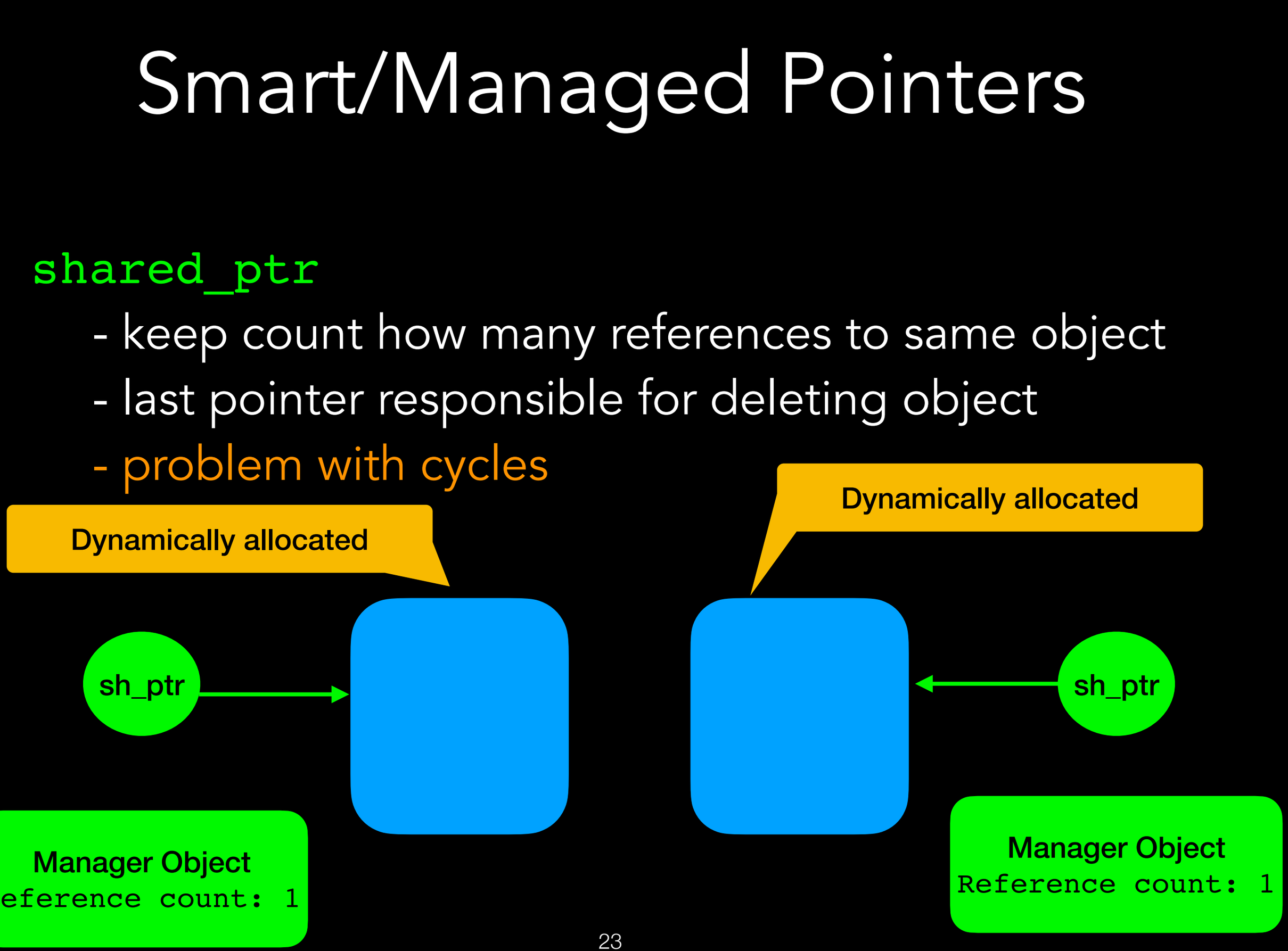
`shared_ptr`

- keep count how many references to same object
- last pointer responsible for deleting object
- problem with cycles

Smart/Managed Pointers

`shared_ptr`

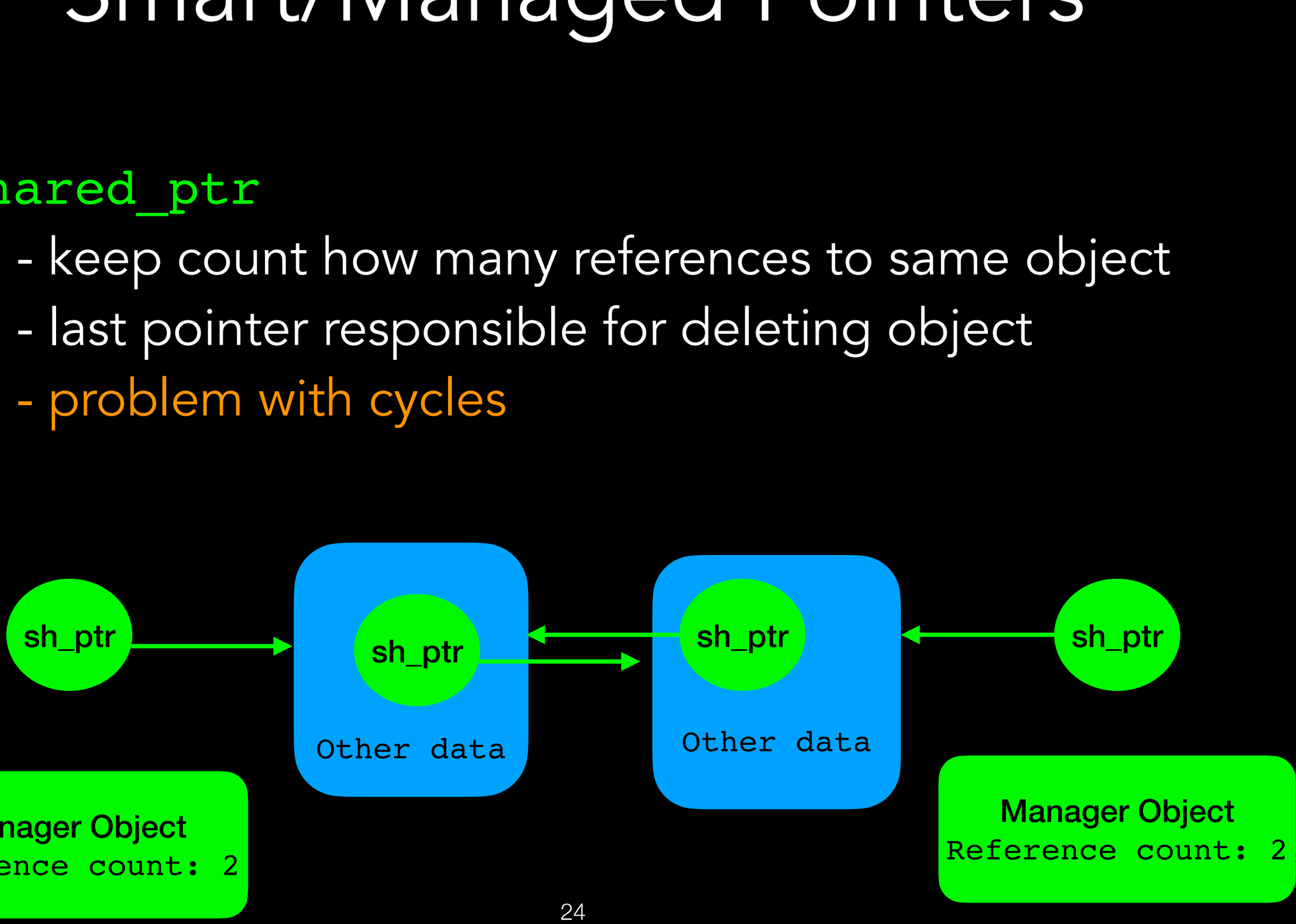
- keep count how many references to same object
- last pointer responsible for deleting object
- problem with cycles



Smart/Managed Pointers

`shared_ptr`

- keep count how many references to same object
- last pointer responsible for deleting object
- problem with cycles

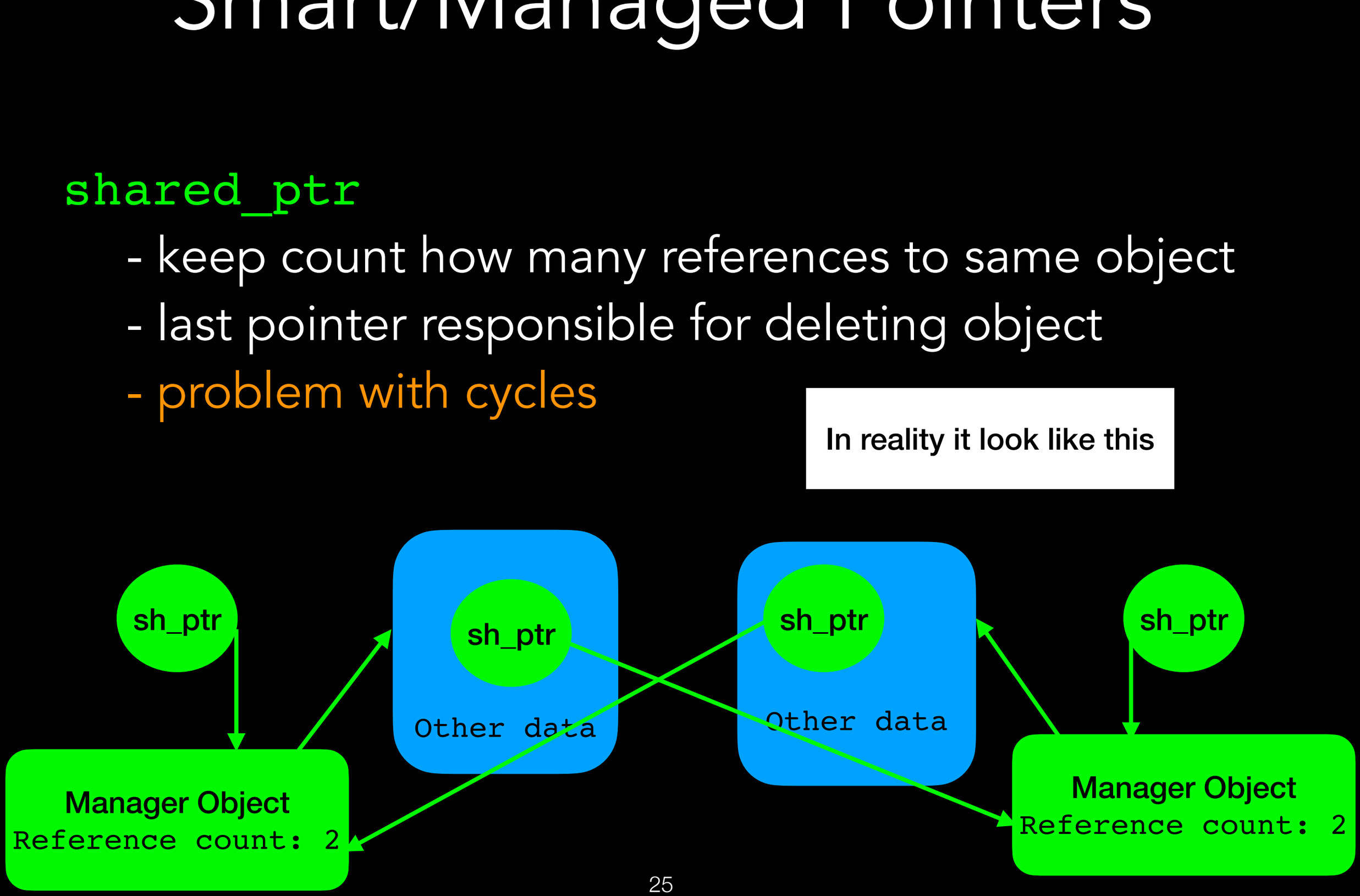


Smart/Managed Pointers

`shared_ptr`

- keep count how many references to same object
- last pointer responsible for deleting object
- problem with cycles

In reality it look like this

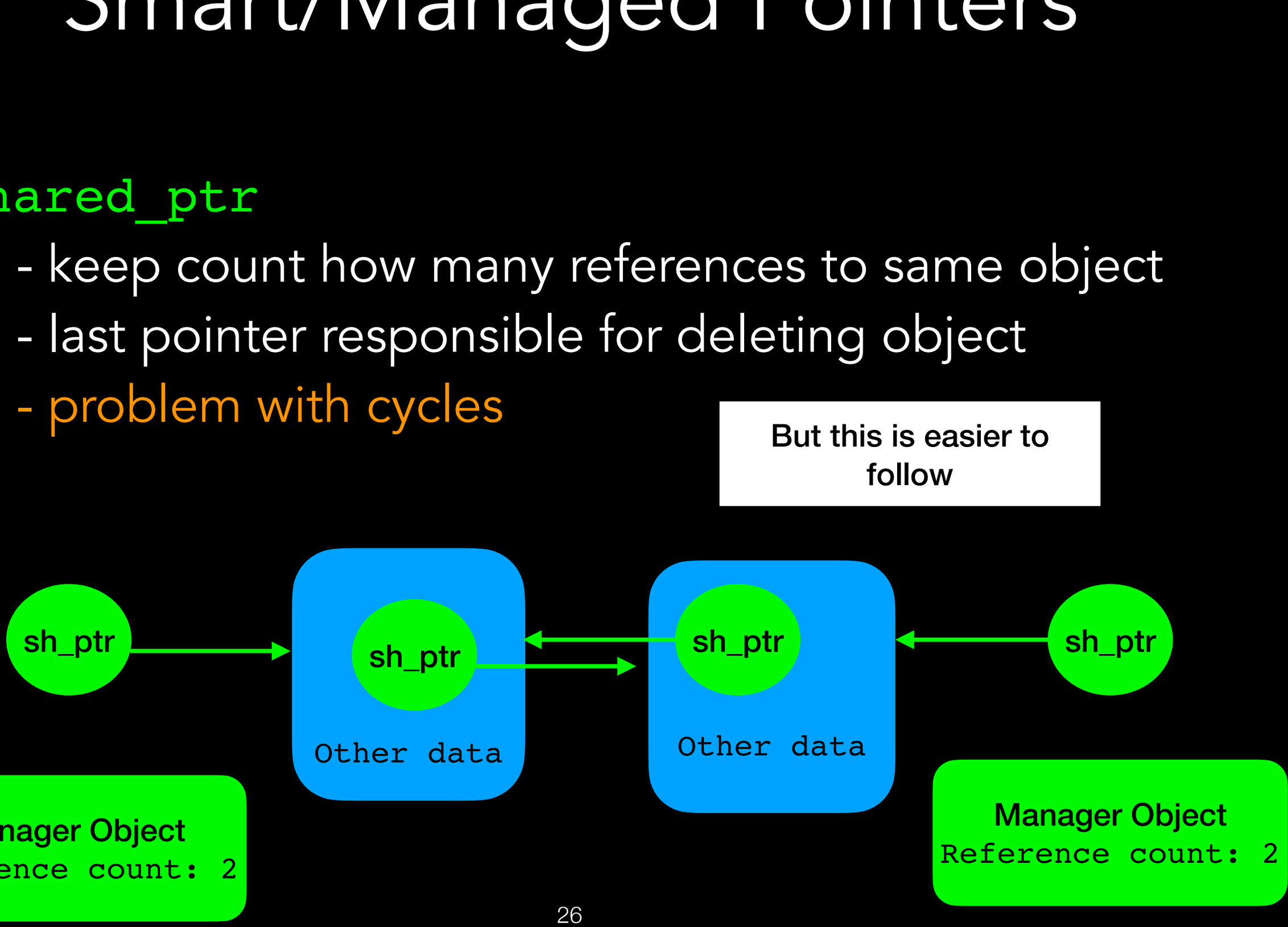


Smart/Managed Pointers

`shared_ptr`

- keep count how many references to same object
- last pointer responsible for deleting object
- problem with cycles

But this is easier to follow

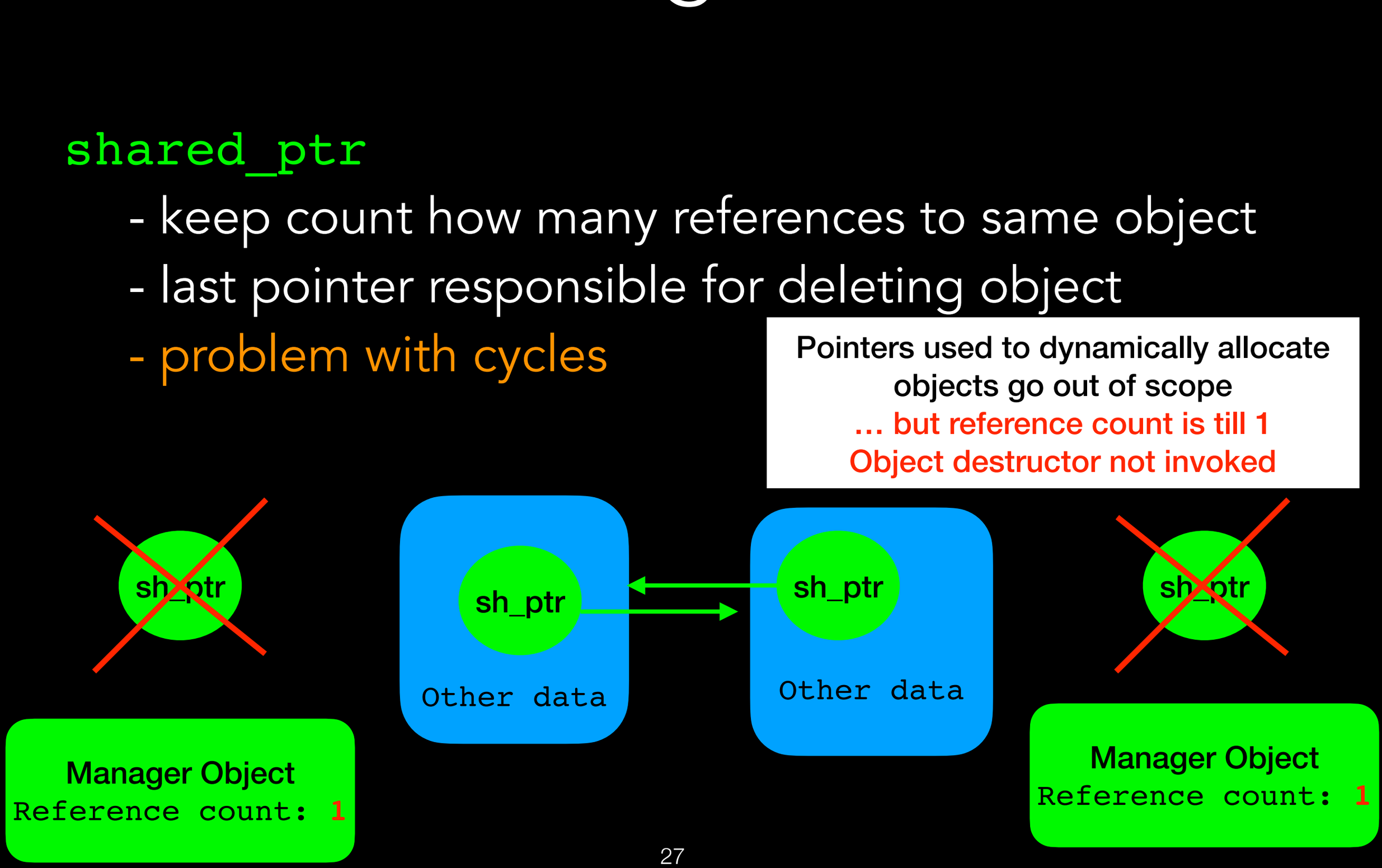


Smart/Managed Pointers

`shared_ptr`

- keep count how many references to same object
- last pointer responsible for deleting object
- problem with cycles

Pointers used to dynamically allocate objects go out of scope
… but reference count is till 1
Object destructor not invoked

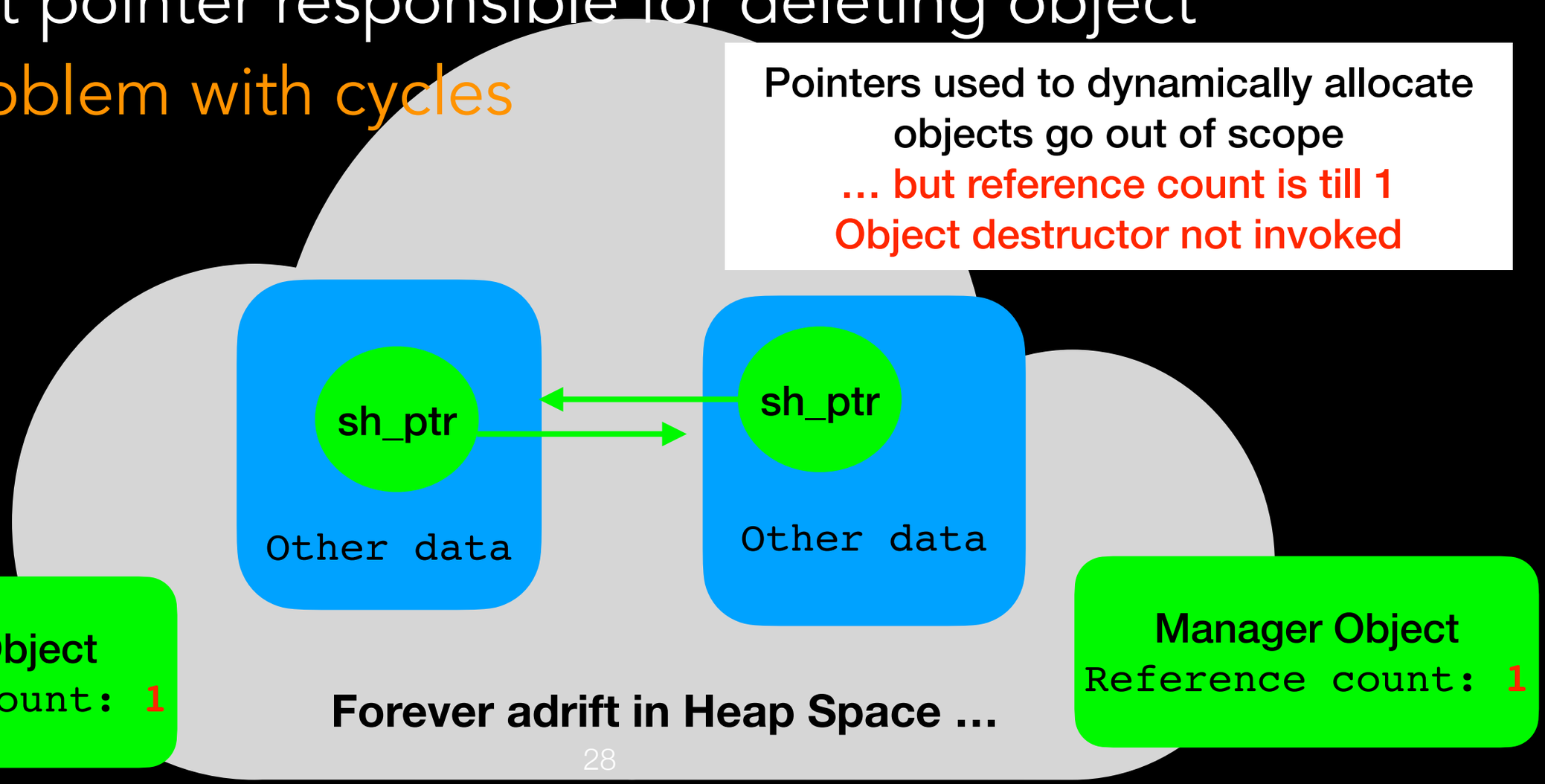


Smart/Managed Pointers

`shared_ptr`

- keep count how many references to same object
- last pointer responsible for deleting object
- problem with cycles

Pointers used to dynamically allocate objects go out of scope
… but reference count is till 1
Object destructor not invoked

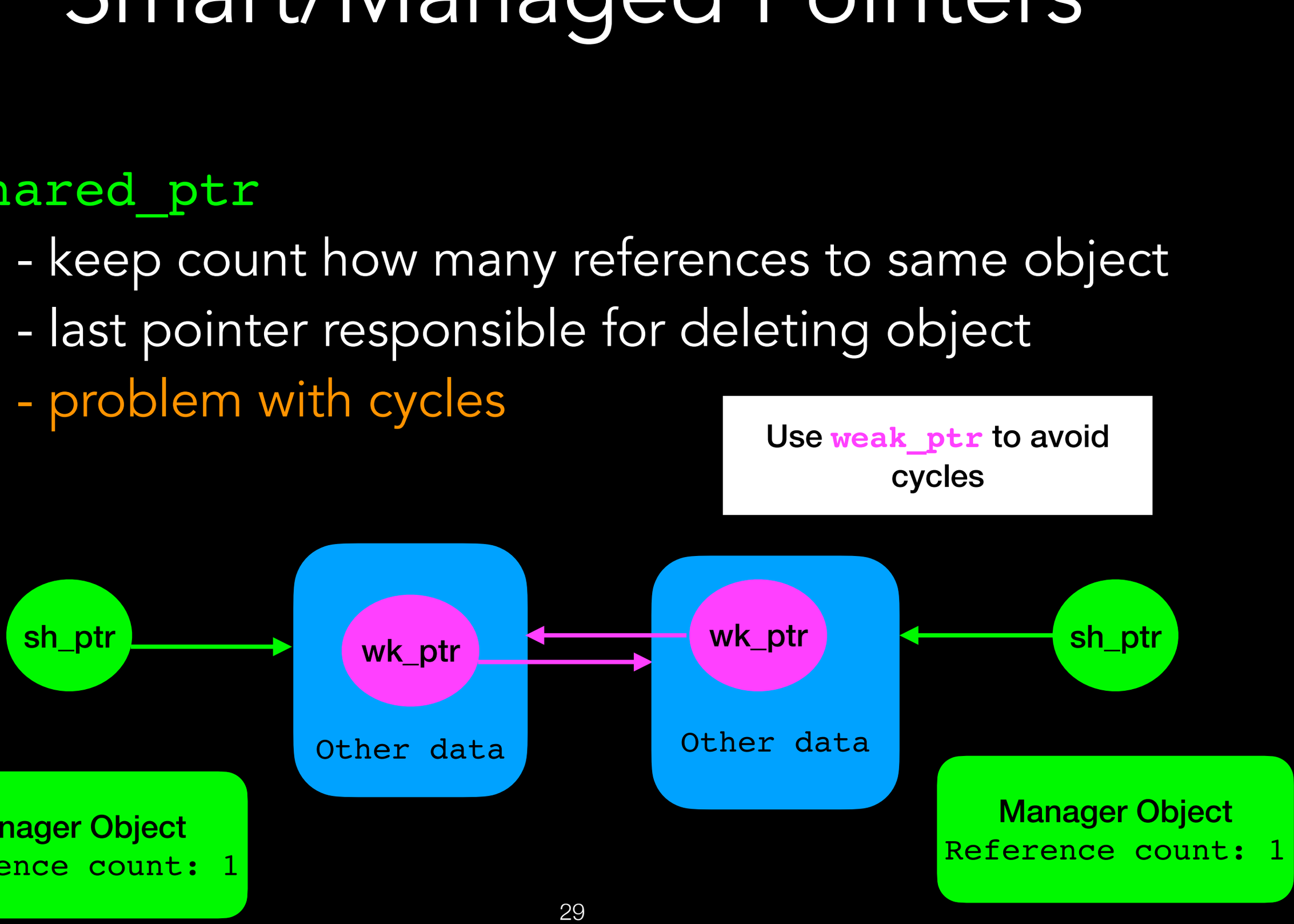


Smart/Managed Pointers

`shared_ptr`

- keep count how many references to same object
- last pointer responsible for deleting object
- problem with cycles

Use `weak_ptr` to avoid cycles

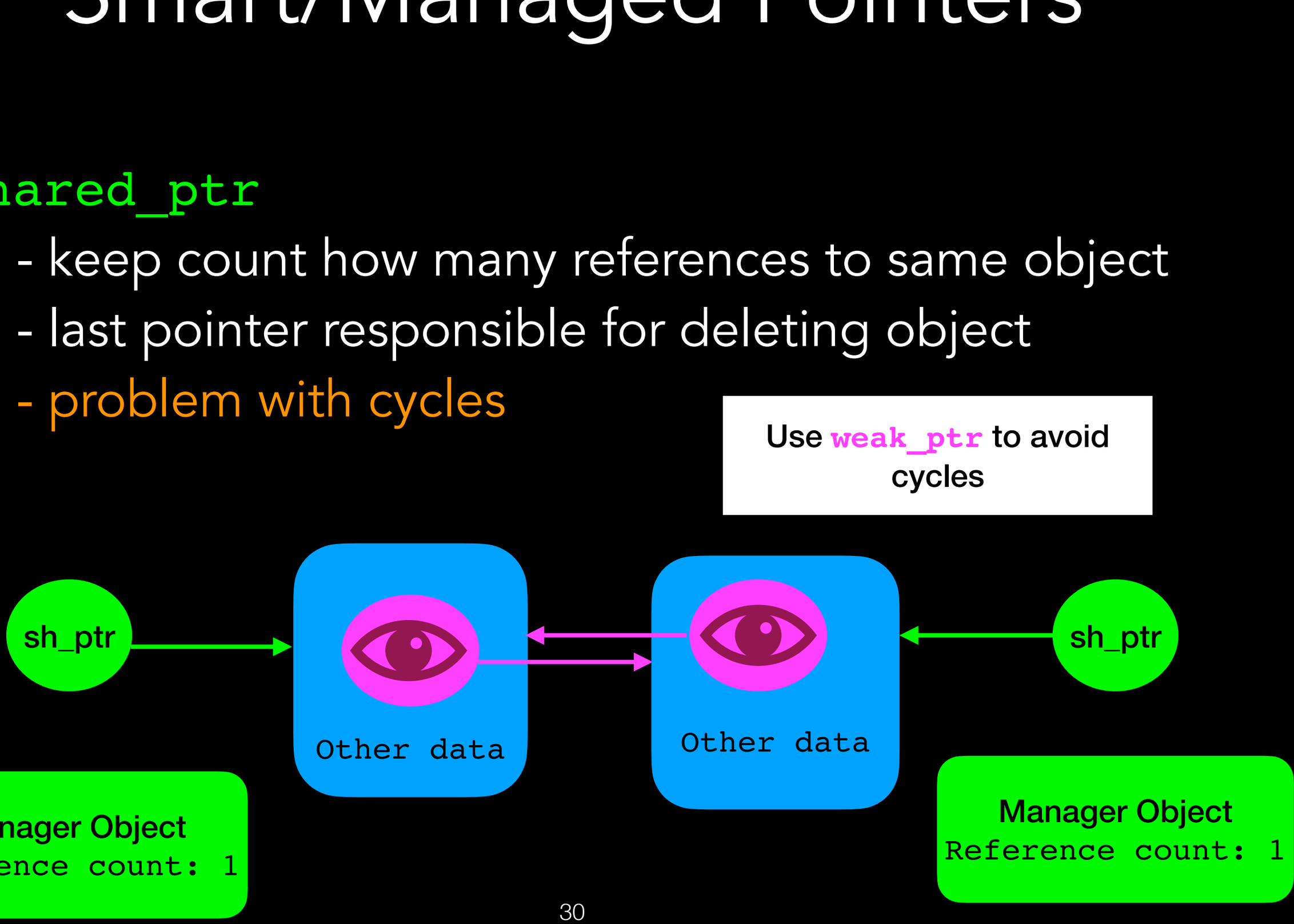


Smart/Managed Pointers

`shared_ptr`

- keep count how many references to same object
- last pointer responsible for deleting object
- problem with cycles

Use `weak_ptr` to avoid cycles

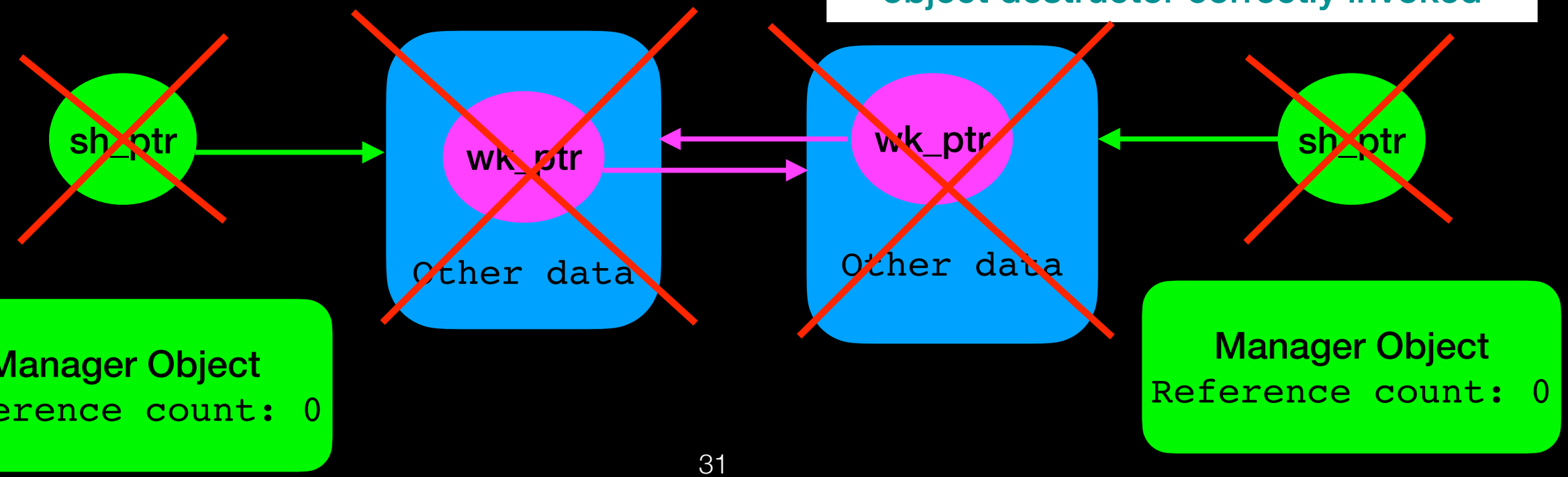


Smart/Managed Pointers

`shared_ptr`

- keep count how many references to same object
- last pointer responsible for deleting object
- problem with cycles

Pointers used to dynamically allocate objects go out of scope
Reference count goes to 0 and object destructor correctly invoked



Syntax

shared_ptr

```
std::shared_ptr<Song> song_ptr1; //declaration only automatically set to nullptr

auto song_ptr2 = std::make_shared<Song>();// equivalent to Song* ptr = new Song
                                          //but creates manager and object in single memory allocation

    // do stuff

std::cout << song_ptr2->getTitle() << std::endl;
```

Syntax

`auto` says: "compiler you figure out the correct type based on what is returned by function on rhs of =

`shared_ptr`

More efficient
Do it this way

```
std::shared_ptr<Song> song_ptr1; //declaration only automatically set to nullptr

auto song_ptr2 = std::make_shared<Song>();// equivalent to Song* ptr = new Song()
                                          //but creates manager and object in single memory allocation

    // do stuff

std::cout << song_ptr2->getTitle() << std::endl;

song_ptr2.reset();
```

Use it just like you would a raw pointer

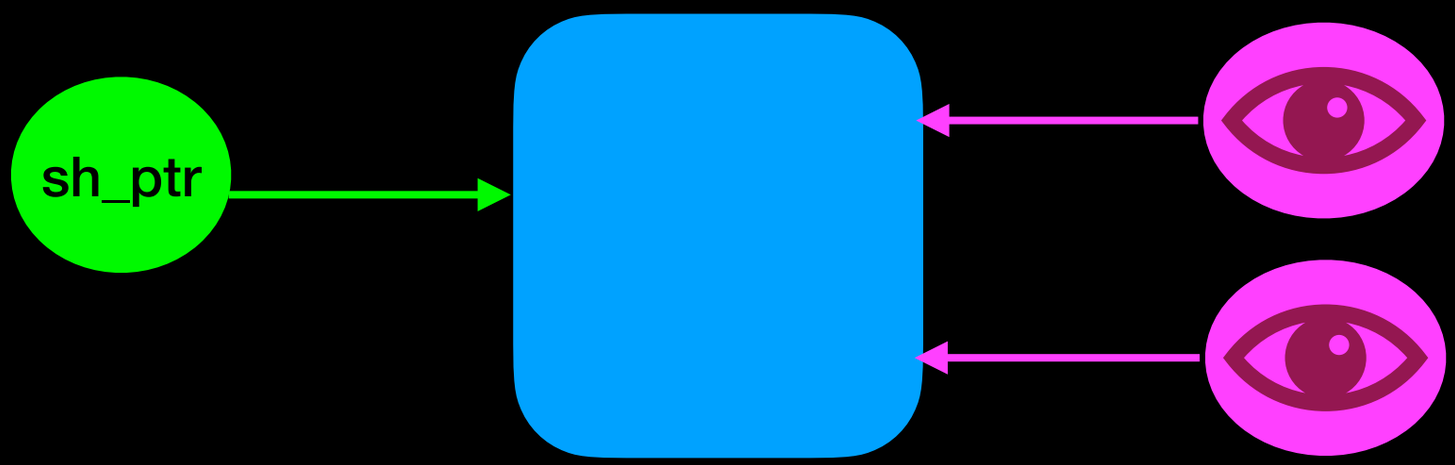
Set the `shared_ptr` to `nullptr` with `reset()` and it will take care of deleting the dynamic object.

Syntax

weak_ptr cannot own object, so cannot be used to allocate a new object — must allocate new object through shared or unique

weak_ptr

```
auto shared_song_ptr = std::make_shared<Song>();

std::weak_ptr<Song> weak_song_ptr1 = shared_song_ptr;
auto weak_song_ptr2 = weak_song_ptr1;
```



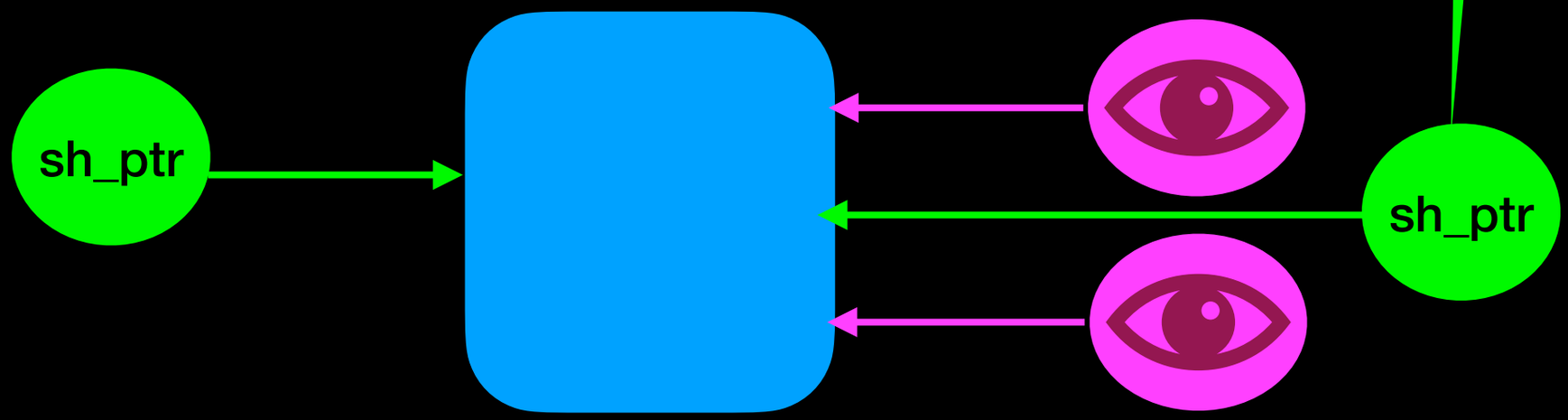
Syntax

`weak_ptr`

```
//cannot directly access object from weak_ptr but can obtain a
//shared_ptr through a weak_ptr
std::shared_ptr<Song> another_shared_ptr =
weak_song_ptr1.lock();
another_shared_ptr->setTitle("my favorite song");

if(weak_song_ptr1.expired())
    //the object has been deleted
```

Obtained with `.lock()`

Returns true if object no longer exists, false otherwise

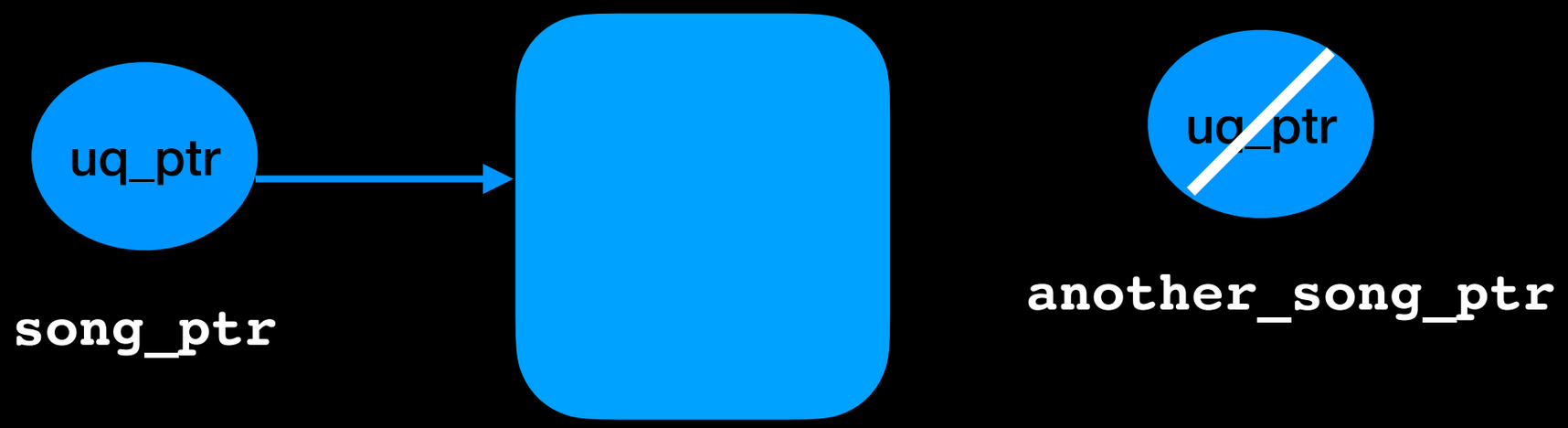


Smart/Managed Pointers

unique_ptr

```
auto song_ptr = std::make_unique<Song>();
std::unique_ptr<Song> another_song_ptr;
                                    //declaration only automatically set to nullptr

another_song_ptr = song_ptr;
                          //ERROR!!! copy assignment not permitted with unique_ptr
```

Error

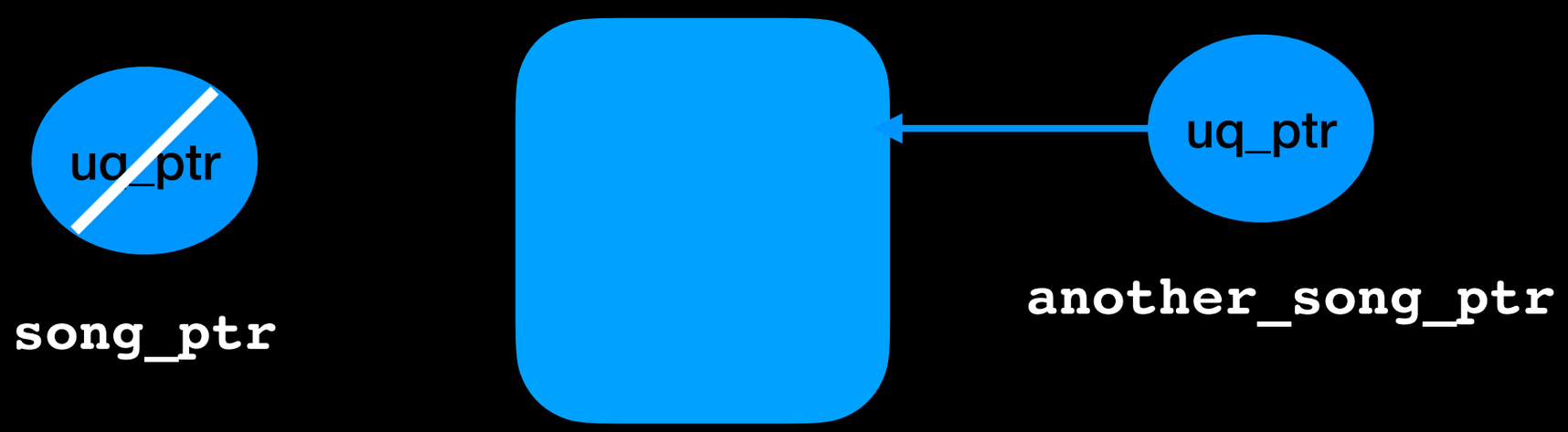


Smart/Managed Pointers

unique_ptr

Correct!

```
auto song_ptr = std::make_unique<Song>();
std::unique_ptr<Song> another_song_ptr;
                                    //declaration only automatically set to nullptr

another_song_ptr = std::move(song_ptr); //CORRECT! but song_ptr is now nullptr
```



In Essence

```
void useRawPointer()
{
   Song* song_ptr = new Song();
   song_ptr->setTitle(“My favorite song”);

   // do stuff

   // don’t forget to delete!!!
   delete song_ptr;
   song_ptr = nullptr;
}
```

Use it just like a raw pointer

It will take care of deleting the object automatically before its own destruction

```
void useSmartPointer()
{
    auto song_ptr = std::make_unique<Song>();
    song_ptr->setTitle("My favorite song");

    // do stuff

} // Song deleted automatically here
```

To summarize

Use smart pointers if you don’t have tight time/space constraints

Beware of cycles when using shared pointers

What's Next?

- ➔ Iterators
- ➔ Finish Sorting
- ➔ Binary Search Trees
- ➔ Balanced Binary Search Trees